

JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 18 OF 18 - PART C OF J

# Advanced Java C: Collections, Streams, and I/O

Collection Semantics, Ordering, Streams, NIO.2, Files, Buffers, and Boundaries

---

FOUNDING AUTHOR

**Vinay Reddy Kalluri**

EDITOR-IN-CHIEF | CHIEF AUDITOR

Choose collections by semantics and cost, reason about ordering and equality, build correct stream reductions, and use I/O boundaries safely.

---

**COLLECTIONS | ORDERING | STREAMS | OPTIONAL | NIO.2 | FILES**

---

**JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION**

Series edition - Release 2026-07-25

Open educational interview-preparation edition

# Start Here - Your Route Through This Volume

**60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to 18B – Advanced Java B – Language. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Part C is the advanced Java-library companion to the DSA volumes, emphasizing API contracts, representation costs, resource ownership, and production boundaries.

## Readiness map

| Decision      | Evidence                                                                                                                                                                                                                |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| START HERE    | Collection semantics matter more than interface names; Ordering and comparator consistency affect correctness; A stream reduction may be invalid in parallel; Bytes, characters, files, and resources cross boundaries. |
| PREPARE FIRST | Volumes 3, 6-8, 11, and 14; Stage 18B language engineering.                                                                                                                                                             |
| MOVE ON WHEN  | Select and size Java collections; Use ordering and navigation APIs safely; Design correct stream and collector pipelines; Handle I/O, files, buffers, and serialization boundaries.                                     |

## Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.



Volume Position

|                                  |                                                                 |                                     |
|----------------------------------|-----------------------------------------------------------------|-------------------------------------|
| 18B - Advanced Java B - Language | <b>YOU ARE HERE</b><br><b>18C - Advanced Java C - Libraries</b> | 18D - Advanced Java D - Concurrency |
|----------------------------------|-----------------------------------------------------------------|-------------------------------------|

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

| CONTENTS NAVIGATION                                                                 |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>Start Here - Your Route Through This Volume</b>                                  | <b>2</b>  |
| <b>Part I - Core Learning</b>                                                       | <b>4</b>  |
| Chapter 1 - Collections Framework Architecture . . . . .                            | 4         |
| Chapter 2 - Java 21 Sequenced Collections . . . . .                                 | 14        |
| Chapter 3 - ArrayList, LinkedList, and List Trade-offs . . . . .                    | 15        |
| Chapter 4 - HashMap, HashSet, and Hashing Internals . . . . .                       | 24        |
| Chapter 5 - TreeMap, TreeSet, Ordering, and Navigable Collections . . . . .         | 35        |
| Chapter 6 - Queues, Deques, PriorityQueue, and Heaps . . . . .                      | 44        |
| Chapter 7 - Comparable, Comparator, Sorting, and Selection . . . . .                | 53        |
| Chapter 8 - Streams, Collectors, Optional, and Spliterators . . . . .               | 63        |
| Chapter 9 - Java I/O, NIO.2, Files, Buffers, and Serialization Boundaries . . . . . | 74        |
| Chapter 10 - Collection Complexity and Selection Matrix . . . . .                   | 85        |
| <b>Part II - Practice and Handoff</b>                                               | <b>92</b> |
| <b>Practice Ladder and Next Steps</b>                                               | <b>92</b> |

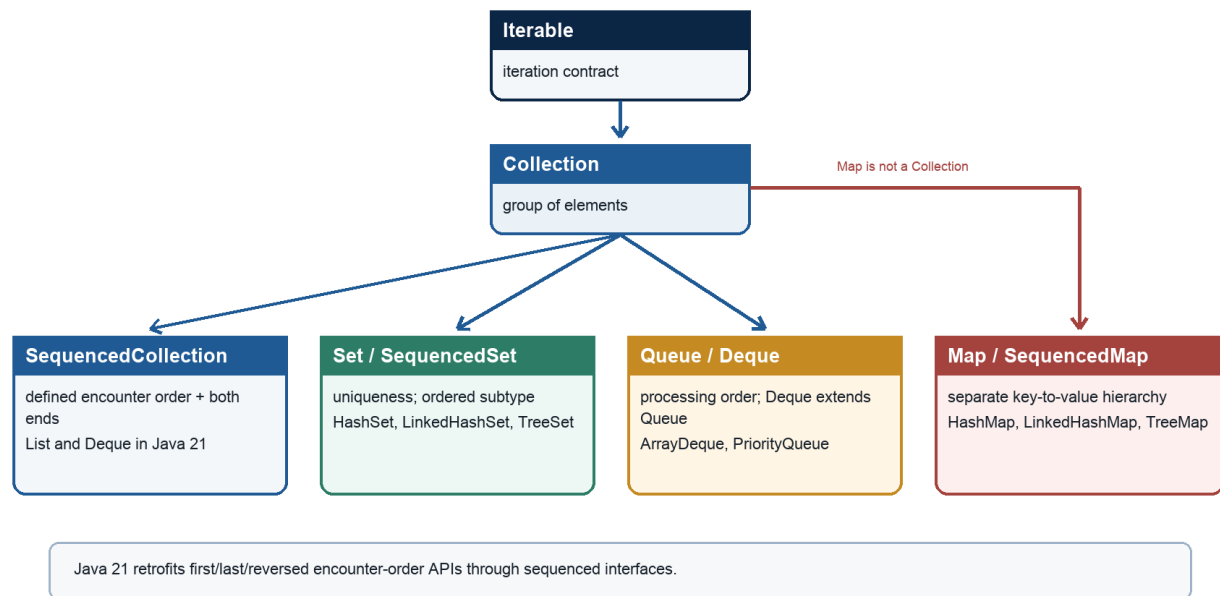
## Part I - Core Learning

## SDE-2 CORE CHAPTER

# Chapter 1 - Collections Framework Architecture

## Collections framework map

Java 21 contracts: select semantics first, then workload and implementation



Java Foundations to Advanced Engineering

*Java 21 collection, sequenced collection, queue, and map contracts*

## Learning objectives

By the end of this chapter, you should be able to:

- distinguish collection interfaces, implementations, algorithms, and views;
- choose between **List**, **Set**, **Queue**, **Deque**, and **Map** from behavioral requirements;
- explain optional operations, iteration order, mutability, and fail-fast iteration;
- reason about structural modification, backed views, equality, and bulk operations;
- state complexity as an implementation property, not an interface promise; and
- design collection-facing APIs that preserve invariants and ownership boundaries.

## WHY IT WORKS | Why this matters at SDE-2

Collections sit in nearly every backend path: request aggregation, caches, indexes, batching, deduplication, scheduling, and persistence mapping. At SDE-2, naming a familiar class is not enough. You are expected to identify the required semantics first, defend complexity claims, understand aliasing, and notice when an innocent view or mutable key creates a correctness defect.

An interview question such as "Which collection would you use?" is usually testing a decision process. Does order matter? Are duplicates allowed? Is lookup by key required? Are nulls valid? Is mutation concurrent? Is a sorted range query needed? The framework gives a vocabulary for those constraints.

## WHY IT WORKS | First-principles model

A collection is an object that owns or exposes a group of references. The Java Collections Framework separates four ideas:

- 1 Interfaces define behavioral contracts, such as uniqueness or positional access.
- 2 Implementations choose a data structure, memory layout, and performance profile.
- 3 Algorithms operate through interfaces, such as sorting or binary search.
- 4 Views expose a live projection of another object, such as a map's key set.

The principal hierarchy is:

### TEXT

```

Iterable
  Collection
    List
    Set
      SortedSet
      NavigableSet
    Queue
    Deque

Map                                (not a Collection)
  SortedMap
  NavigableMap
  
```

**Map** is separate because it models key-value associations rather than individual elements. Its `keySet()`, `values()`, and `entrySet()` methods bridge into the **Collection** hierarchy through views.

**Specification boundary:** Interface contracts specify observable behavior. They generally do not promise array storage, linked nodes, hashing strategy, tree shape, amortized cost, or fail-fast detection. Complexity claims belong to the documentation of a concrete implementation.

## Core terminology

- **Element:** A reference stored by a collection. Collections do not contain primitive values directly; boxing creates wrapper objects when needed.
- **Structural modification:** A change that can alter iteration structure, usually adding or removing elements. Replacing a value may or may not be structural for a particular implementation.
- **Encounter order:** The order in which an iteration mechanism presents elements. Some collections define it; others do not.
- **Natural ordering:** Ordering defined by `Comparable`.
- **Optional operation:** An interface method that an implementation may reject with `UnsupportedOperationException`.
- **View:** A projection backed by another object. Changes may be visible in both directions.
- **Snapshot:** An independent representation of state at a point in time.
- **Unmodifiable:** Mutation through that reference is rejected. It does not necessarily mean the underlying data or elements are immutable.
- **Immutable:** State cannot change after construction, including through aliases permitted by the abstraction.
- **Fail-fast iterator:** An iterator that attempts to detect unsupported concurrent structural modification and throw `ConcurrentModificationException`.

## Detailed mechanics

### Selecting the abstraction

Use a `List` when position, encounter order, or duplicates are meaningful. Use a `Set` for membership and uniqueness. Use a `Queue` when processing order matters and elements enter and leave through queue operations. Use a `Deque` for both ends or stack behavior. Use a `Map` when a unique key identifies a value.

Program parameters and return types to the narrowest useful interface. A method that only iterates can accept `Iterable<T>` or `Collection<T>` rather than `ArrayList<T>`. Do not hide an important semantic property, however: accepting `Set<T>` communicates uniqueness, while accepting `Collection<T>` does not.

## Optional operations and capability

`Collection` includes methods such as `add`, `remove`, and `clear`, but implementations can reject them. `List.of(...)` produces an unmodifiable list. `Arrays.asList(array)` is fixed-size: `set` works, but size-changing methods do not. This design lets algorithms use a common interface, but it means the static type alone does not prove mutability.

Java has no standard type-level distinction between mutable and read-only collections. Document ownership and capability explicitly. Prefer immutable copies at public boundaries when callers should not mutate state:

JAVA

```
final class RoutingTable {
    private final List<String> backends;

    RoutingTable(Collection<String> backends) {
        this.backends = List.copyOf(backends);
    }

    List<String> backends() {
        return backends;
    }
}
```

`List.copyOf` also rejects null elements. That is useful only if null rejection matches the domain contract.

## Iterators and structural modification

An `Iterator` is a cursor with `hasNext`, `next`, and optional `remove`. Enhanced `for` normally uses it. Removing directly from a typical collection during iteration can invalidate cursor state:

JAVA

```
for (String id : ids) {
    if (id.isBlank()) {
        ids.remove(id); // usually wrong
    }
}
```

Use `Iterator.remove`, `removeIf`, or a separate result collection. The iterator's own removal operation updates both the structure and its bookkeeping.

**HotSpot note:** Common OpenJDK collections maintain a modification counter and compare it with an iterator's expected value. This is implementation detail and detection is best-effort.

`ConcurrentModificationException` is a bug signal, not a synchronization mechanism. Exact fields and detection points are version-sensitive.

## Views and aliasing

`map.keySet()`, `map.values()`, and `map.entrySet()` are normally backed views. Removing a key from the key set removes the mapping. An entry's `setValue` can update the map when supported.

`list.subList(from, to)` is also a backed view. A structural change to the parent outside the sublist can make later sublist operations fail.

Wrappers from `Collections.unmodifiableList(source)` are read-only views, not copies. If another alias mutates `source`, the wrapper reflects it. By contrast, `List.copyOf(source)` returns an unmodifiable list whose membership is not backed by later source changes. Neither operation freezes mutable element objects.

## Equality and bulk operations

`List.equals` is order-sensitive. `Set.equals` compares membership independent of order. `Map.equals` compares mappings. These contracts enable equality across implementations: an `ArrayList` can equal a `LinkedList` with the same sequence.

Bulk methods include `addAll`, `removeAll`, `retainAll`, `containsAll`, `removeIf`, and `replaceAll`. Their apparent single-call form does not imply constant time. Cost depends on both receiver and argument. For example, removing every element found in an `ArrayList` argument can be quadratic when membership checks are linear. Converting the lookup side to a `HashSet` can improve the expected bound.

## Null policy and element validity

Null support varies. Some general-purpose collections allow null; many queues, concurrent collections, sorted structures with natural ordering, and factory-created immutable collections reject it. A robust API validates domain values before selecting an implementation rather than relying on an incidental null policy.



## TRACE IT | Worked Java example

The following method groups unique order IDs by customer while preserving first-seen customer order and first-seen order ID order:

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

record Order(String id, String customerId) {}

final class OrderIndex {
    static Map<String, List<String>> build(Collection<Order> orders) {
        Map<String, Set<String>> working = new LinkedHashMap<>();

        for (Order order : orders) {
            if (order == null || order.id() == null || order.customerId() == null) {
                throw new IllegalArgumentException("order fields must be non-null");
            }
            working.computeIfAbsent(
                order.customerId(), ignored -> new LinkedHashSet<>())
```

JAVA (continued 2/2)

```
                .add(order.id());
        }

        Map<String, List<String>> result = new LinkedHashMap<>();
        working.forEach((customer, ids) ->
            result.put(customer, List.copyOf(ids)));
        return Map.copyOf(result);
    }

    public static void main(String[] args) {
        List<Order> input = new ArrayList<>();
        input.add(new Order("o-1", "c-7"));
        input.add(new Order("o-2", "c-8"));
        input.add(new Order("o-1", "c-7"));
        input.add(new Order("o-3", "c-7"));

        System.out.println(build(input));
    }
}
```

One subtlety: `Map.copyOf` guarantees an unmodifiable map, but its iteration order is not specified to preserve the insertion order of the supplied `LinkedHashMap`. If public iteration order is part of the result contract, return an unmodifiable copy with an order-preserving implementation, for example `Collections.unmodifiableMap(new LinkedHashMap<>(result))`, and document the guarantee.

## TRACE IT | Execution or memory walkthrough

For the four inputs above:

- 1 `o-1/c-7` creates the first map entry and a set containing `o-1`.
- 2 `o-2/c-8` creates a second entry. The map's encounter order is now `c-7`, `c-8`.
- 3 The duplicate `o-1/c-7` reaches the existing set. `Set.add` returns `false`; membership remains unchanged.
- 4 `o-3/c-7` appends a second unique ID to the first customer's insertion-ordered set.
- 5 The conversion creates new lists, so clients cannot add or remove IDs through the result.

The working representation uses a map object, map entries, set objects, set entries, and references to existing strings. The result adds list storage and map entries. Peak memory includes both representations until the method returns and the working map becomes unreachable. The code copies collection structure, not `String` content; strings are immutable, so sharing them is safe.

## Complexity and performance

Let  $n$  be the number of orders and  $u$  the number of unique customer-order pairs. With typical `LinkedHashMap` and `LinkedHashSet` behavior, building has expected  $O(n)$  time and  $O(u)$  space; producing the result costs  $O(u)$ . Hash collisions, resizing, and key methods affect constants and worst cases.

Interface-only complexity should be stated cautiously:

| Operation                    | Interface guarantee     | Common implementation example                                               |
|------------------------------|-------------------------|-----------------------------------------------------------------------------|
| <code>List.get(i)</code>     | No bound                | <code>ArrayList</code> : $O(1)$ ; <code>LinkedList</code> : $O(n)$          |
| <code>Set.contains(x)</code> | No bound                | <code>HashSet</code> : expected $O(1)$ ; <code>TreeSet</code> : $O(\log n)$ |
| <code>Map.get(k)</code>      | No bound                | <code>HashMap</code> : expected $O(1)$ ; <code>TreeMap</code> : $O(\log n)$ |
| iteration                    | Semantic order may vary | Usually $O(n)$ , but hash-table capacity can affect traversal               |
| <code>containsAll</code>     | No bound                | Depends on sizes and membership cost                                        |

Memory is often as important as asymptotic time. Node-based structures add per-element objects and pointers. Array-based structures may reserve unused capacity but provide locality. Measure representative workloads rather than extrapolating from Big-O alone.

## WATCH OUT | Edge cases and common mistakes

- Choosing a concrete type before defining uniqueness, ordering, and access semantics.
- Returning a mutable internal list and unintentionally granting write access.
- Assuming an unmodifiable wrapper is a snapshot or that immutable membership makes elements immutable.
- Depending on `HashMap` or `HashSet` iteration order.
- Modifying a collection directly while iterating over it.
- Treating `ConcurrentModificationException` as guaranteed detection of a race.
- Forgetting that `subList` and map collection views are backed by their owner.
- Passing a collection to itself in a bulk operation whose behavior is undefined or surprising.
- Using mutable objects whose `equals`, `hashCode`, or ordering fields change while stored.
- Assuming every implementation permits null or mutation.
- Calling `size()` repeatedly on a nonstandard collection without checking whether it is cheap.
- Confusing `Collection` with `Collections`; the latter is an algorithm and wrapper utility class.

## Production engineering notes

Define collection contracts in API documentation: order, duplicates, null policy, mutability, snapshot versus live view, thread safety, and expected scale. A return type of `List<T>` communicates sequence but not ownership.

Defensively copy inputs when the component must own stable membership. For very large inputs, copying can be costly; an explicit ownership-transfer convention or immutable persistent data structure may be more appropriate. Never expose a lazily changing view across layers unless that behavior is deliberate.

Avoid sharing ordinary mutable collections across threads without a synchronization policy. A synchronized wrapper makes individual calls mutually exclusive, but compound actions such as "check then add" still need one lock scope. Concurrent collections have different iteration and atomicity contracts and should be selected explicitly.

Prefer domain operations over raw collection exposure. `routingTable.chooseBackend()` protects invariants better than `routingTable.backends().remove(0)`. Validate capacity and input size where untrusted requests can cause large allocations. Instrument unusually large collection sizes and expensive conversions in latency-sensitive paths.

## TRY IT | Interview questions and model answers

### Why is `Map` not a subtype of `Collection`?

A collection models individual elements, while a map models key-value mappings with key uniqueness. `Map` exposes collection views for keys, values, and entries, but operations such as adding one arbitrary element do not fit its contract.

### What does fail-fast mean?

It means an iterator may detect an unsupported structural modification and throw `ConcurrentModificationException` promptly. It is best-effort behavior in common implementations, not a thread-safety guarantee or a correctness mechanism.

### What is the difference between an unmodifiable view and an immutable copy?

An unmodifiable view rejects mutation through that reference but can reflect mutations through another alias. An immutable copy has independent, unchangeable membership. In both cases, referenced element objects can still be mutable unless separately constrained.

### Why accept an interface rather than `ArrayList`?

It minimizes coupling and admits any implementation satisfying the required behavior. The chosen interface should still express semantics: `Set` is preferable to `Collection` when uniqueness is required.

### Can interface type tell you complexity?

Usually no. `List.get` can be constant or linear depending on implementation. State the concrete type and assumptions whenever making a complexity claim.

### How do backed views affect correctness?

They create aliases. Mutating the owner can change the view, and supported mutations through the view can change the owner. They are useful for efficient projections but dangerous if callers assume snapshots.

## TRY IT | Exercises

- 1 Design a method signature for returning active feature flags in deterministic order without allowing callers to mutate membership. State every contract not captured by the type.
- 2 Predict the result of removing an entry through `map.entrySet().iterator().remove()` and explain why it differs from mutating the map directly during iteration.
- 3 Compare `Collections.unmodifiableList(source)`, `List.copyOf(source)`, and `new ArrayList<>(source)` for mutability, null handling, and aliasing.
- 4 Rewrite a quadratic `removeAll` workflow so membership tests use a set. Give time and space bounds.
- 5 Create a small program that demonstrates a `subList` backed view. Then structurally modify the parent outside the view and record the observed behavior without treating it as a portable synchronization technique.

## RECAP | Chapter summary

The Collections Framework separates behavioral interfaces from data-structure implementations, reusable algorithms, and backed views. A sound choice begins with semantics: order, uniqueness, lookup, mutation, concurrency, and ownership. Complexity belongs to concrete implementations. Iterators, views, and optional operations make the framework flexible, but they also create important capability and aliasing boundaries. Production APIs should state those boundaries explicitly and copy or wrap data according to an intentional ownership model.

## TRY IT | Revision checklist

- ☐ I can sketch the **Collection** hierarchy and explain why **Map** is separate.
- ☐ I can choose among **List**, **Set**, **Queue**, **Deque**, and **Map** from requirements.
- ☐ I distinguish interface contracts from implementation complexity.
- ☐ I understand optional operations and common fixed-size or unmodifiable collections.
- ☐ I can explain structural modification and correct iterator removal.
- ☐ I can distinguish a view, an unmodifiable wrapper, a shallow copy, and deep immutability.
- ☐ I know the equality semantics of lists, sets, and maps.
- ☐ I document order, nulls, ownership, thread safety, and expected scale in APIs.



## SDE-2 CORE CHAPTER

## Chapter 2 - Java 21 Sequenced Collections

---

### Sequenced collections

Java 21 adds `SequencedCollection`, `SequencedSet`, and `SequencedMap` to express defined encounter order and operations at both ends.

Representative methods:

**JAVA**

```
list.getFirst();
list.getLast();
list.addFirst(value);
list.reversed();

map.firstEntry();
map.lastEntry();
map.sequencedKeySet();
map.reversed();
```

Support and mutation behavior still depend on the concrete collection. A reversed view is generally a view, not an independent copy. An unmodifiable collection remains unmodifiable through its reversed view.

## SDE-2 CORE CHAPTER

## Chapter 3 - ArrayList, LinkedList, and List Trade-offs

### Learning objectives

By the end of this chapter, you should be able to:

- explain the `List` contract and its positional model;
- derive the invariants of dynamic arrays and doubly linked lists;
- analyze access, insertion, removal, iteration, and memory costs;
- explain amortized growth without claiming a fixed growth formula;
- use list iterators, sublists, immutable factories, and array conversions safely; and
- select a list implementation from workload and locality, not folklore.

### WHY IT WORKS | Why this matters at SDE-2

Lists look simple, which makes them good interview probes. A candidate who says "linked lists are better for insertion" without discussing how the insertion position is found misses the important cost. Backend systems usually favor `ArrayList` because indexed storage, cache locality, low allocation count, and predictable iteration dominate. `LinkedList` remains useful for learning pointer invariants and can serve as a deque, but it is rarely the best default list.

At SDE-2 you should reason from operation distribution and data size. A batch assembled once and scanned many times has different needs from a cursor that repeatedly inserts at a known position. You should also recognize that a list can be modifiable, fixed-size, unmodifiable, or a backed range view despite sharing the same `List` type.

### WHY IT WORKS | First-principles model

A `List<E>` represents an ordered sequence indexed from `0` through `size - 1`. It permits duplicates and, depending on implementation, may permit null. Equality compares corresponding elements in sequence.

An array-backed list maintains conceptually:

#### TEXT

```
elements: [e0, e1, e2, e3, null, null, null, null]
size:      4
capacity:  8
```

```
Invariant: 0 <= size <= capacity
Valid logical elements occupy indexes [0, size).
```

A doubly linked list maintains nodes:

#### TEXT

```

null <- [prev|A|next] <-> [prev|B|next] <-> [prev|C|next] -> null
      first                                last

```

Invariant: `first.prev == null`, `last.next == null`,  
and adjacent next/prev links agree.

The dynamic array makes location computation cheap: address is derived from base plus index. The linked representation must follow references to find an index. Once a linked node is already known, relinking neighbors is constant work, but the public `List` API usually supplies an index or search value, not a node handle.

**Specification boundary:** `List` promises positional behavior, not storage representation or complexity. `ArrayList` documents constant-time positional access and amortized constant-time append. Its exact capacity-growth policy is not a public contract.

## Core terminology

- **Logical size:** Number of elements visible through the list.
- **Capacity:** Number of slots currently available in an array-backed representation.
- **Amortized cost:** Average cost per operation over a sequence, including occasional expensive operations.
- **Random access:** Efficient access by arbitrary index. `RandomAccess` is a marker used by some algorithms to adapt behavior.
- **Locality:** Tendency for nearby data to be located near each other in memory and used together.
- **Node:** A linked-list record containing an element and neighbor references.
- **Cursor:** A position between elements, represented by `ListIterator`.
- **Range view:** A live portion of a list, typically from `subList`.
- **Fixed-size list:** Element replacement is allowed, but size-changing operations are rejected.

## Detailed mechanics

### ArrayList operations

`get(i)` and `set(i, value)` validate the index and access one slot. Appending writes into the next slot if capacity remains. When full, the implementation allocates a larger array and copies references. Inserting at index `i` shifts the suffix `[i, size)` one slot right. Removing at `i` shifts the later suffix left and clears the obsolete final slot so the removed object is not retained through that array reference.

Suppose size equals capacity and append triggers a copy of  $n$  references. One append is  $O(n)$ , but if capacity grows geometrically, many preceding appends were cheap. Across a long append sequence, total copied references form a geometric series bounded by a constant multiple of  $n$ , yielding amortized  $O(1)$  append.

`ensureCapacity` can reduce repeated resizing when a reasonable size estimate exists. `trimToSize` can reduce spare storage but may cost  $O(n)$  and cause future regrowth. Neither should be called reflexively.

**HotSpot note:** OpenJDK has historically grown `ArrayList` capacity by roughly one-half of the old capacity in common cases, with boundary handling. This is version-sensitive implementation detail. Never write logic that depends on a particular capacity sequence.

## LinkedList operations

Each element is stored in a node with references to predecessor and successor. Adding at either end updates a small number of links and endpoints. Finding index  $i$  walks from the nearer end in common implementations, still  $O(n)$  asymptotically. Removing by value also searches before unlinking.

`LinkedList` implements both `List` and `Deque`. Its strongest use is often end-based queue behavior, but `ArrayDeque` usually offers better locality and lower allocation overhead for that role. A linked list does not reserve unused array capacity, yet it usually uses much more memory per element because every element has a node object and two link fields.

## Iteration and ListIterator

For both structures, one full iterator traversal is  $O(n)$ . Indexed traversal can be disastrous for a linked list:

JAVA

```
for (int i = 0; i < list.size(); i++) {  
    consume(list.get(i));  
}
```

If `list` is a `LinkedList`, repeated `get` makes the loop  $O(n^2)$ . An enhanced `for` loop or iterator is  $O(n)$ .

`ListIterator` supports forward and backward traversal, reports neighboring indexes, and can add, remove, or replace at its cursor when supported. Its state rules matter: `remove` or `set` must follow a successful `next` or `previous`, and cannot immediately follow `add` or another `remove`.

Typical general-purpose list iterators are fail-fast when they detect structural mutation outside the iterator. Detection is best-effort and must not be used for synchronization or program control. Use the iterator's mutation methods or establish exclusive ownership.

## List construction variants

- `new ArrayList<>()` is mutable and resizable.
- `new ArrayList<>(source)` is a shallow mutable copy.
- `Arrays.asList(array)` is a fixed-size view backed by the array. `set` changes the array.
- `List.of(elements...)` is unmodifiable and rejects null.
- `List.copyOf(source)` is unmodifiable and rejects null; it may reuse an already suitable immutable instance.
- `Collections.unmodifiableList(source)` is an unmodifiable view backed by `source`.
- `stream.toList()` returns an unmodifiable list, but code should not assume a concrete implementation.

## Sublists

`list.subList(from, to)` uses a half-open range: `from` is included and `to` is excluded. It is normally backed by the parent. Clearing a sublist can efficiently remove a range from the parent. Structural modification of the backing list outside the view invalidates assumptions; subsequent behavior is generally not useful to depend upon and commonly produces `ConcurrentModificationException`.

If a stable independent range is needed, use `new ArrayList<>(list.subList(from, to))`.

## Conversion to arrays

`toArray()` returns `Object[]`. `toArray(new String[0])` and `toArray(String[]::new)` produce a typed array. Modern JVMs optimize common zero-length-array idioms, so prefer clarity and measure if this conversion is actually hot. Conversion copies references and costs  $O(n)$  space and time.

## TRACE IT | Worked Java example

This example keeps a sorted timeline in an `ArrayList` and inserts a new event using binary search:

JAVA (continued 1/2)

```
import java.time.Instant;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

record Event(Instant at, String id) {}

final class Timeline {
    private static final Comparator<Event> ORDER =
        Comparator.comparing(Event::at).thenComparing(Event::id);

    private final ArrayList<Event> events = new ArrayList<>();

    void add(Event event) {
        int result = java.util.Collections.binarySearch(events, event, ORDER);
        int insertionPoint = result >= 0 ? result : -result - 1;
        events.add(insertionPoint, event);
    }
}
```

JAVA (continued 2/2)

```
List<Event> between(Instant startInclusive, Instant endExclusive) {
    ArrayList<Event> result = new ArrayList<>();
    for (Event event : events) {
        if (event.at().isBefore(startInclusive)) {
            continue;
        }
        if (!event.at().isBefore(endExclusive)) {
            break;
        }
        result.add(event);
    }
    return List.copyOf(result);
}

List<Event> snapshot() {
    return List.copyOf(events);
}
}
```

The list invariant is that `events` is sorted by `ORDER`. The class, not callers, owns mutation. A real system with frequent arbitrary insertion may choose a tree index or append plus periodic sort; the example illustrates how search cost and movement cost differ.



## TRACE IT | Execution or memory walkthrough

Assume the list contains events at (10:00, a), (10:05, b), and (10:20, c). Adding (10:12, d) performs binary search:

### TEXT

```
low=0 high=2, mid=1 -> 10:05 is smaller, low=2
low=2 high=2, mid=2 -> 10:20 is larger, high=1
search ends; insertion point=2
```

```
before: [10:00/a, 10:05/b, 10:20/c, empty]
shift:  [10:00/a, 10:05/b, 10:20/c, 10:20/c]
write:  [10:00/a, 10:05/b, 10:12/d, 10:20/c]
```

Binary search uses  $O(\log n)$  comparisons, but insertion shifts  $O(n)$  references in the worst case. Overall insertion remains  $O(n)$ . This distinction is a common interview trap.

The array holds references, not inline `Event` record payloads. `snapshot()` creates another list storage object containing the same immutable `Event` references. Because `Event` contains immutable components, shallow sharing is safe. If elements were mutable, the snapshot would protect membership only.

## Complexity and performance

| Operation                                 | <code>ArrayList</code> | <code>LinkedList</code>          |
|-------------------------------------------|------------------------|----------------------------------|
| get or set by index                       | $O(1)$                 | $O(n)$                           |
| append                                    | amortized $O(1)$       | $O(1)$                           |
| add or remove at front                    | $O(n)$                 | $O(1)$                           |
| insert/remove at arbitrary index          | $O(n)$ shift           | $O(n)$ search plus $O(1)$ relink |
| insert/remove through positioned iterator | $O(n)$ shift           | $O(1)$ relink                    |
| search by value                           | $O(n)$                 | $O(n)$                           |
| full iteration                            | $O(n)$                 | $O(n)$                           |
| auxiliary structure per element           | array slot             | node plus links                  |

The table hides hardware effects. `ArrayList` traversal is usually faster because reference slots are contiguous, copying can use optimized bulk operations, and there are fewer allocations and indirections. `LinkedList` nodes increase garbage collection work and can miss CPU caches.

For small lists, constants dominate. For very large lists, the `int` index and array-size limits matter. Neither implementation should be treated as a disk-scale container. Benchmark representative data with the intended JDK and hardware.

## WATCH OUT | Edge cases and common mistakes

- Using `LinkedList` for frequent indexed reads, creating accidental quadratic behavior.
- Claiming linked insertion is  $O(1)$  while ignoring the  $O(n)$  search for a position.
- Using `remove(1)` on `List<Integer>` when intending to remove the value `1`; it removes index `1`. Use `remove(Integer.valueOf(1))` for value removal.
- Assuming `Arrays.asList` is a normal resizable `ArrayList`.
- Exposing `subList` as a durable independent result.
- Forgetting half-open range boundaries and accepting `from > to` or invalid indexes.
- Retaining a tiny sublist backed by a very large custom or older representation; make an independent copy when retention is uncertain.
- Sorting a list and later mutating fields used by ordering without restoring the invariant.
- Treating `List.of(array)` as a list of array elements when a primitive array is passed; a primitive array is one reference element.
- Forgetting null rejection in immutable factories.
- Relying on exact `ArrayList` capacity growth.
- Using copy-on-write lists for write-heavy workloads; that concurrent implementation copies on mutation.

## Production engineering notes

Default to `ArrayList` for general mutable sequence storage. Provide an expected capacity when reliable batch metadata exists, but cap estimates derived from untrusted input to avoid oversized allocation. Prefer `ArrayDeque` for stack and queue operations. Use `LinkedList` only when its measured workload and cursor or end operations justify its overhead.

Preserve invariants behind an API. If a list must remain sorted, do not return the mutable list. If duplicates are invalid, a `Set` may express the model better. If lookups by ID dominate, maintain a map rather than repeatedly scanning a list.

Bulk construction is frequently better than incremental middle insertion: append, validate, sort once, and publish an immutable snapshot. Keep lists local to a request when possible. Shared mutation requires synchronization or a concurrent design; an unmodifiable wrapper does not make concurrent reads safe while another alias mutates the backing list.

Watch memory retention in queues or batch buffers. Clear references when custom array structures remove elements. Standard implementations do this, but application-level lists can still retain large element graphs simply because a long-lived owner retains the list.

## TRY IT | Interview questions and model answers

### Why is `ArrayList` usually faster than `LinkedList` for iteration?

Its reference slots are contiguous, so it uses fewer objects and indirections and has better cache locality. Both are  $O(n)$ , but their constants and memory behavior differ greatly.

### Is insertion into `LinkedList` constant time?

Relinking is constant time once the node or iterator position is known. `add(index, value)` must locate the index and is therefore  $O(n)$  overall.

### How can `append` be both $O(n)$ and amortized $O(1)$ ?

An `append` that triggers `resize` copies existing references and costs  $O(n)$ . Geometric capacity growth makes `resizes` infrequent, so total work across many `appends` is linear and average cost per `append` is constant.

### What does `subList` return?

A backed range view, not an independent copy. Supported mutations affect the parent, and external structural changes to the parent can invalidate the view.

### What is the difference between `size` and `capacity`?

`Size` is the number of logical elements. `Capacity` is allocated slot count in an array-backed implementation. `Capacity` is not part of the `List` contract.

### When would you deliberately choose `LinkedList`?

Only when a workload relies on constant-time end operations or `insert/remove` at a maintained iterator position and measurement shows it is appropriate. Even for `deque` use, I would compare `ArrayDeque` first.

## TRY IT | Exercises

- 1 Dry-run removal at index 2 from an array list of six elements. Count moved references and identify the slot that must be cleared.
- 2 Prove that indexed traversal of a linked list is  $O(n^2)$  by summing traversal distances.
- 3 Implement a sorted list insertion method that places a duplicate after all equal values. State why binary search alone does not promise which equal item it finds.
- 4 Demonstrate aliasing between an array and `Arrays.asList(array)`. Contrast it with `new ArrayList<>(...)`.
- 5 Design a benchmark hypothesis comparing `ArrayList`, `LinkedList`, and `ArrayDeque` for removing from the front. Include allocation and warmup considerations.
- 6 Refactor a class that returns its mutable internal list into an ownership-safe API while preserving deterministic order.

## RECAP | Chapter summary

`List` defines ordered positional semantics, while implementations determine representation and cost.

`ArrayList` uses a resizable reference array, offering constant-time indexed access, amortized constant-time append, efficient traversal, and suffix movement for middle changes. `LinkedList` uses doubly linked nodes, offering constant-time relinking at known positions but linear positional access and high allocation overhead. Views, factories, and iterators introduce mutability and aliasing differences that must be part of API design.

## TRY IT | Revision checklist

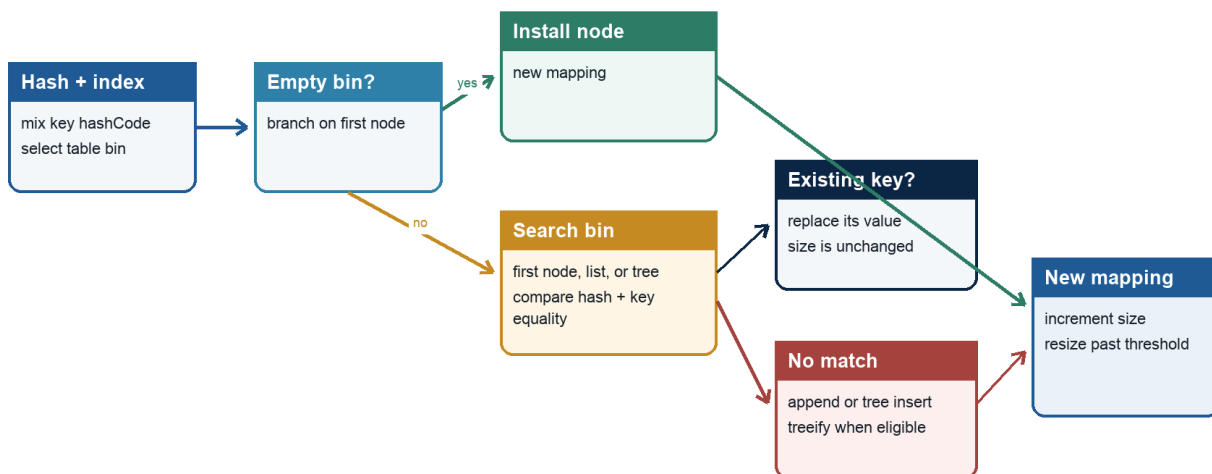
- ☐ I can state dynamic-array and doubly linked-list invariants.
- ☐ I can derive amortized append cost without relying on a growth factor.
- ☐ I include position-finding cost when analyzing linked insertion.
- ☐ I avoid indexed traversal for sequential-access lists.
- ☐ I know the behavior of `Arrays.asList`, `List.of`, `List.copyOf`, and unmodifiable wrappers.
- ☐ I can use `ListIterator` statefully and safely.
- ☐ I understand that `subList` is a backed half-open range.
- ☐ I can choose a list using access pattern, locality, allocation, and ownership.

## SDE-2 CORE CHAPTER

# Chapter 4 - HashMap, HashSet, and Hashing Internals

## HashMap put path

Representative OpenJDK-style mechanics; not a full API contract



Equal keys need equal hash codes; equality-relevant key state must remain stable while stored.  
Expected  $O(1)$  depends on distribution; collision structure and thresholds are version-sensitive.

Java Foundations to Advanced Engineering

*Representative HashMap insertion branches and correctness prerequisites*

## Learning objectives

By the end of this chapter, you should be able to:

- derive hash-table lookup from equality and bucket selection;
- state the `equals` and `hashCode` contract and key-stability invariant;
- explain collisions, load factor, resizing, and expected complexity;
- describe a typical OpenJDK `HashMap` without presenting it as a platform guarantee;
- use `compute`, `merge`, views, and iteration safely; and
- identify security, concurrency, memory, and API-design risks around hash tables.

## WHY IT WORKS | Why this matters at SDE-2

Hash maps support indexes, caches, joins, deduplication, frequency counts, and graph representations. They are also a rich interview subject because correctness spans language-level equality, data-structure invariants, amortized analysis, and mutation discipline.

At SDE-2, "lookup is  $O(1)$ " is incomplete. The useful answer is expected  **$O(1)$**  under a suitable hash distribution and stable key behavior, with resizing and collision caveats. You should be able to diagnose a missing entry caused by a mutated key, distinguish absence from a null value, and choose an ordered, identity-based, weak-key, concurrent, or sorted alternative when semantics demand it.

## WHY IT WORKS | First-principles model

A hash table transforms a key into a candidate region rather than comparing it with every stored key. Conceptually:

TEXT

```
hashCode(key) -> mixed hash -> bucket index -> inspect candidates -> equals
```

The hash narrows the search; `equals` determines logical key identity. Different keys can have the same hash and must coexist as a collision. Equal keys must be directed to the same search region.

The central invariant is:

TEXT

```
For every stored entry (k, v), lookup using a key equal to k  
must search the bucket containing that entry.
```

That requires equal objects to have equal hash codes and requires fields used by equality and hashing to remain stable while the key is stored.

`HashSet<E>` uses the same membership idea without an application value. A typical implementation is backed by a hash map whose keys are set elements and whose values are a private marker.

**Specification boundary:** `Map` specifies unique keys and mapping behavior. `HashMap` specifies null support and general performance expectations in its API documentation, but bucket array shape, hash mixing, resize thresholds, tree bins, and constants are implementation details.

## Core terminology

- **Hash code:** An `int` computed for an object and used to group possible matches.
- **Collision:** Distinct, non-equal keys assigned to the same bucket.
- **Bucket/bin:** A table position containing zero or more entries.
- **Capacity:** Number of table buckets in a typical hash-table representation.
- **Load factor:** Ratio controlling how full the table may become before resizing.
- **Threshold:** Entry count that triggers resizing, often capacity times load factor.
- **Rehash/resizing:** Allocating a larger table and redistributing entries.
- **Hash flooding:** Many keys deliberately or accidentally colliding.
- **Tree bin:** A collision bucket represented by a balanced tree in some implementations.
- **Key stability:** Requirement that equality and hash behavior not change while a key is stored.
- **Canonical key:** Stable value representation used as a map key, such as a normalized immutable identifier.

## Detailed mechanics

### Equality and hashing contract

`Object` establishes these practical rules:

- 1 If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` must be true.
- 2 Unequal objects may share a hash code.
- 3 Repeated hash calls during one execution should be consistent while equality-relevant state is unchanged.

A record automatically derives value-based `equals` and `hashCode` from components, making records useful keys when components are themselves stable:

```
JAVA
```

```
record TenantUser(String tenantId, String userId) {}
```

If a mutable class uses `tenantId` and `userId` for hashing, changing either after insertion can strand the entry. The object remains physically in its old bucket while lookup computes a new bucket.

### Lookup, insertion, and collision resolution

A typical `get(key)` computes a hash, maps it to an index, then checks candidate entries. It first compares hash values and then key equality. Correct comparisons account for identical references and null according to implementation policy.



On `put(key, value)`, a matching key replaces its value and returns the old value. If no matching key exists, a new entry is linked or otherwise inserted into the bucket. Map size changes only for a new key. A `HashSet.add` similarly returns `false` when an equal element is already present.

Many hash tables use separate chaining: each bucket holds multiple entry nodes. Other hash tables use open addressing, but ordinary OpenJDK `HashMap` has traditionally used bucket chains with tree conversion under certain conditions.

## Capacity, load factor, and resizing

A low load factor uses more buckets and generally shortens collision searches. A high load factor conserves table space but increases collisions. The default is intended as a general compromise, not a universal optimum.

When size crosses a threshold, a typical implementation allocates a larger bucket array and relocates or splits entries. The resize is  $O(n)$ , but geometric growth makes a long sequence of inserts expected amortized  $O(1)$  each. Pre-sizing can avoid repeated growth when an estimate is reliable. Excessive pre-sizing wastes memory and can slow full iteration because iteration may scan buckets as well as entries.

**HotSpot note:** In current OpenJDK implementations, capacities are generally powers of two, index selection uses masked hash bits, and resizing often doubles capacity. Hash spreading mixes high bits into low bits. Details and helper methods can change by JDK release.

## Tree bins

Long collision chains degrade lookup toward  $O(n)$ . Modern OpenJDK versions can transform a sufficiently populated bin into a red-black tree when the table is also large enough, giving  $O(\log n)$  comparison depth for that bin under suitable conditions. Bins can later return to list form.

**HotSpot note:** Treeification and untreeification thresholds, minimum table capacity, tie-breaking, and node layouts are version-sensitive. The Java API does not guarantee tree bins or worst-case logarithmic lookup. Treat them as resilience in a particular implementation, not as permission to use poor hashes.

## Null, absence, and defaulting

`HashMap` permits one null key and multiple null values. Therefore `get(key) == null` can mean no mapping or a mapping to null. Use `containsKey` when the distinction matters. Prefer avoiding null values in domain maps when absence has a clear meaning.

`getOrDefault` returns the mapped value even if that value is null; it uses the default only when no mapping exists. `putIfAbsent` treats a null mapping as absent for its operation. Read each method's contract carefully.

## Compute and merge methods

`computeIfAbsent` is ideal for multimap assembly:

JAVA

```
index.computeIfAbsent(customerId, ignored -> new ArrayList<>()).add(order);
```

The mapping function should be short and should not structurally mutate the same map recursively. If it returns null, no mapping is recorded. `computeIfPresent` runs only for a present non-null mapping. `compute` considers both presence and absence; a null result removes the mapping. `merge(k, incoming, remapper)` inserts `incoming` when absent or combines it with the current non-null value; a null remapping result removes the key.

These methods make a compound update concise on an ordinary map but do not make that map thread-safe. Concurrent map implementations define stronger per-key atomic behavior for their overrides.

## Views and iteration

`keySet`, `values`, and `entrySet` are backed views. Iterate `entrySet` when both key and value are needed; repeated `map.get(key)` is redundant. Removing through a view removes mappings. A `HashMap` defines no stable iteration order. Even if a run appears consistent, a resize, JDK change, input change, or hash change can alter it.

Typical iterators are fail-fast under detected unsupported structural modification. Replacing the value for an existing key is often nonstructural, but this is not a general concurrency contract. `Map.Entry` objects from iteration may be tied to the map and should not be retained as permanent independent records; use `Map.entry(k, v)` or a domain record for a snapshot pair.

## TRACE IT | Worked Java example

This frequency index uses an immutable composite key and `merge`:

JAVA

```
import java.util.HashMap;
import java.util.List;
import java.util.Map;

record Endpoint(String method, String path) {
    public Endpoint {
        if (method == null || path == null) {
            throw new IllegalArgumentException("endpoint fields are required");
        }
        method = method.toUpperCase(java.util.Locale.ROOT);
    }
}

record Request(String method, String path) {}

final class RequestCounter {
    static Map<Endpoint, Long> count(List<Request> requests) {
        Map<Endpoint, Long> counts = new HashMap<>();
        for (Request request : requests) {
            Endpoint key = new Endpoint(request.method(), request.path());
            counts.merge(key, 1L, Long::sum);
        }
        return Map.copyOf(counts);
    }

    public static void main(String[] args) {
        var requests = List.of(
            new Request("get", "/health"),
            new Request("GET", "/orders"),
            new Request("GET", "/health"));
        System.out.println(count(requests));
    }
}
```

Normalization is part of key construction, so equality semantics match the domain rule that HTTP method case is irrelevant here. The path is intentionally not normalized; whether `/orders/` equals `/orders` is an application decision, not a hash-table concern.

`Map.copyOf` prevents result membership changes and rejects null keys or values. It does not promise the iteration order of the source hash map, so output formatting and tests must not depend on order.

## TRACE IT | Execution or memory walkthrough

For the sample input:

- 1 `Endpoint("get", "/health")` becomes `("GET", "/health")`. Its hash selects a bucket. No equal key exists, so `merge` inserts count 1.
- 2 `("GET", "/orders")` selects some bucket and is inserted with 1.
- 3 A new `Endpoint("GET", "/health")` is a different reference but is record-equal to the first key and has the same hash. Lookup reaches the same bucket, `equals` succeeds, and `Long::sum` produces 2.

Conceptually, with capacity eight:

### TEXT

```
bucket 0: empty
bucket 1: [Endpoint(GET,/orders) -> 1]
bucket 2: empty
bucket 3: [Endpoint(GET,/health) -> 2]
bucket 4: empty
...
```

Actual indexes are deliberately omitted because hash spreading and table state are implementation-sensitive. The map stores one entry per unique endpoint, not one per request. Temporary equal key objects created for repeated endpoints become unreachable after lookup. The boxed `Long` values are replaced as counts grow because `Long` is immutable.

Now consider a broken mutable key:

### JAVA

```
final class MutableKey {
    String id;
    MutableKey(String id) { this.id = id; }
    public boolean equals(Object other) {
        return other instanceof MutableKey k && id.equals(k.id);
    }
    public int hashCode() { return id.hashCode(); }
}
```

After `map.put(key, value)`, changing `key.id` changes its lookup hash. `map.get(key)` can return null even though iteration still reveals the same reference. This violates the table's key-stability assumption, not the map's implementation.

## Complexity and performance

For  $n$  entries and a reasonable hash distribution:

| Operation          | Expected                         | Collision-degraded      | Notes                                             |
|--------------------|----------------------------------|-------------------------|---------------------------------------------------|
| get, put, remove   | $O(1)$                           | up to $O(n)$ abstractly | tree bins may improve a particular implementation |
| containsKey        | $O(1)$                           | up to $O(n)$            | invokes hash/equality work                        |
| full iteration     | $O(n + \text{capacity})$ typical | same form               | oversized tables can hurt                         |
| resize             | $O(n)$                           | $O(n)$                  | amortized across insertions                       |
| HashSet membership | same as backing hash table       | same caveat             | value is only a marker                            |

Hash computation itself may not be constant in key size. `String.hashCode` is proportional to string length on first computation in many implementations, though caching and compact-string representation are implementation concerns. Expensive `equals` also increases collision cost.

Space is  $O(n + \text{capacity})$  plus entry/node overhead. Load factor changes the trade-off between empty buckets and collision depth. `LinkedHashMap` adds ordering links. `IdentityHashMap` uses reference identity and a specialized representation. `EnumMap` uses enum ordinals and is typically compact. Choose semantics first, then measure.

## WATCH OUT | Edge cases and common mistakes

- Overriding `equals` without a consistent `hashCode`.
- Mutating equality-relevant key state after insertion.
- Assuming unequal keys require different hash codes.
- Assuming iteration order or using printed `HashMap` order in golden tests.
- Using `get(k) == null` to prove absence when null values are permitted.
- Calling `containsValue` as though it were a hashed lookup; it generally scans values.
- Performing expensive or recursive side effects in `computeIfAbsent`.
- Assuming ordinary `HashMap.compute` is atomic across threads.
- Pre-sizing from an untrusted claimed count and allocating excessive memory.
- Using arrays as value keys without content-based wrappers; arrays inherit identity equality.
- Returning mutable lists stored as map values and calling the outer map immutable.
- Expecting tree bins to fix adversarial equality or expensive hash functions.
- Sharing a `HashMap` for concurrent mutation without synchronization.
- Forgetting that `HashSet` uniqueness is exactly the equality definition of its elements.

## Production engineering notes

Prefer immutable, small, domain-specific keys. Normalize once at the boundary and make normalization rules explicit. Avoid concatenated string keys when components can be represented by a record; concatenation risks ambiguity and repeated allocation.

Size maps from credible estimates, accounting for load factor rather than setting initial capacity equal to expected entries and assuming no resize. Avoid massive speculative capacities. Monitor cache/index cardinality, eviction, and skew; an unbounded map is often a memory leak by design.

For user-controlled keys, use current supported JDKs and validate request sizes. Hash flooding can convert an expected constant-time endpoint into CPU exhaustion. Tree bins help in common OpenJDK versions but do not replace input limits, timeouts, and observability.

Use `LinkedHashMap` when encounter or access order is a contract, `TreeMap` for ordered range queries, `ConcurrentHashMap` for shared concurrent mutation, `EnumMap` for enum keys, and identity maps only when reference identity is truly the model. A synchronized wrapper still requires external locking around compound iteration or multi-call invariants.

When publishing maps, specify whether values are deeply immutable. `Map.copyOf` freezes mappings, not mutable objects reachable from values. Be cautious with cached negative or null results; explicit wrapper types often make state clearer.

## TRY IT | Interview questions and model answers

### How does `HashMap.get` work?

It computes a key hash, derives a candidate bucket, then checks entries in that bucket using hash and equality. Expected lookup is constant with good distribution; collisions require additional comparisons. Exact bucket and tree mechanics are implementation-specific.

### Why must equal objects have equal hash codes?

The table uses the hash to choose where to search. If equal keys could select different regions, lookup could miss a logically identical stored key.

### What happens when a key changes after insertion?

If equality or hashing changes, lookup searches using the new hash while the entry remains placed according to the old hash. Retrieval and removal can fail. Use immutable keys or stable identity fields.

### What is a collision? Is it an error?

A collision is two unequal keys sharing a bucket or hash result. It is normal and must be resolved by equality checks. Excessive collisions hurt performance.

### Why is `HashMap` lookup not simply guaranteed $O(1)$ ?

The API does not guarantee collision distribution, hash cost, or a particular bucket representation. Expected  $O(1)$  assumes suitable hashing and load. Worst-case abstract lookup can be linear.

### How is `HashSet` commonly implemented?

It is commonly backed by a `HashMap`, storing set elements as keys and a shared private marker as the map value. That is an OpenJDK-style implementation fact, while the set contract only guarantees uniqueness.

### When would you use `computeIfAbsent`?

For lazy per-key initialization such as grouping values into lists. The function should be short, return the desired initial value, and avoid structurally modifying the same map.

## TRY IT | Exercises

- 1 Implement a correct immutable `Coordinate` key manually with `equals` and `hashCode`, then compare it with a record.
- 2 Dry-run three colliding keys through insert, replacement, lookup, and removal.
- 3 Write a program that demonstrates the mutable-key failure. Repair it by making the key immutable.
- 4 Build a word-frequency map using `merge`, then return entries sorted by frequency without relying on hash iteration order.
- 5 Given one million expected entries, explain how load factor, key size, entry overhead, and capacity affect memory. Identify what must be measured rather than guessed.
- 6 Compare an outer `Map.copyOf` containing mutable lists with a deeply unmodifiable result.

## RECAP | Chapter summary

Hash tables narrow key search through a hash and confirm identity through `equals`. Their correctness depends on consistent equality and hashing plus stable keys. Collision handling, capacity, load factor, and resizing produce expected constant-time operations under ordinary assumptions, not an unconditional guarantee. OpenJDK's power-of-two tables and tree bins are useful implementation knowledge but are version-sensitive. Production design requires bounded cardinality, deliberate key semantics, explicit order and null policies, and a concurrency strategy.

## TRY IT | Revision checklist

- ☐ I can state the `equals` and `hashCode` contract.
- ☐ I can derive lookup, insertion, collision handling, and resizing.
- ☐ I explain expected and amortized costs with assumptions.
- ☐ I know why mutable keys become unreachable by normal lookup.
- ☐ I distinguish absence from a null mapping.
- ☐ I can use `merge` and compute methods with their null semantics.
- ☐ I do not depend on `HashMap` or `HashSet` iteration order.
- ☐ I label tree bins, thresholds, hash spreading, and capacity shape as implementation details.
- ☐ I can select ordered, concurrent, enum, identity, or sorted alternatives by semantics.



## SDE-2 CORE CHAPTER

# Chapter 5 - TreeMap, TreeSet, Ordering, and Navigable Collections

## Learning objectives

By the end of this chapter, you should be able to:

- distinguish sorted, encounter-ordered, and unordered collections;
- explain binary-search-tree and red-black-tree invariants;
- use `TreeMap`, `TreeSet`, `NavigableMap`, and `NavigableSet` operations correctly;
- reason about comparator consistency, mutable keys, ranges, and backed views;
- analyze logarithmic operations and ordered traversal; and
- choose a tree-based collection when navigation is required, not merely for deterministic output.

## WHY IT WORKS | Why this matters at SDE-2

Ordered maps and sets support time-window indexes, floor and ceiling lookup, leaderboards, routing ranges, and nearest-neighbor decisions. Interviews often ask for "the next event," "all keys in a range," or "the largest value no greater than x." A hash table cannot answer those efficiently without an additional scan or sort.

The difficult part is not recalling `TreeMap`. It is preserving an ordering that is total, stable, and consistent with the application's notion of identity. A comparator that treats two distinct keys as equal silently changes map membership. A range view that escapes its intended lifetime introduces aliasing. An SDE-2 engineer recognizes these as correctness contracts.

## WHY IT WORKS | First-principles model

A binary search tree stores one entry per node and maintains:

### TEXT

```
all keys in left subtree < node key
all keys in right subtree > node key
```

Here `<` is defined by a comparator or natural ordering. An ordinary binary search tree can become a chain under sorted insertion, producing linear operations. A balanced tree adds structural invariants that keep height logarithmic.

A red-black tree conceptually enforces:

- 1 Every node has a color, red or black.
- 2 The root is black.
- 3 Missing leaf sentinels are black.
- 4 A red node has no red child.
- 5 Every path from a node to a missing leaf contains the same number of black nodes.

These rules limit the longest root-to-leaf path to at most about twice the shortest, so height is  $O(\log n)$ . Insertions and removals restore invariants using recoloring and rotations.

**Specification boundary:** `TreeMap` and `TreeSet` promise sorted and navigable behavior with documented logarithmic basic operations. The API does not require a red-black tree specifically. OpenJDK's representation, node fields, and balancing code are implementation details.

## Core terminology

- **Natural ordering:** Ordering provided by a type's `Comparable.compareTo`.
- **Comparator:** Object defining an external ordering through `compare(a, b)`.
- **Total order:** Every pair can be compared consistently, with transitivity and antisymmetry-like sign rules.
- **Ordering-equivalent:** `compare(a, b) == 0`.
- **Consistent with equals:** Ordering-equivalent exactly when `a.equals(b)` for relevant values.
- **Floor:** Greatest element less than or equal to a target.
- **Lower:** Greatest element strictly less than a target.
- **Ceiling:** Least element greater than or equal to a target.
- **Higher:** Least element strictly greater than a target.
- **Range view:** Backed portion bounded by keys or elements.
- **Rotation:** Local tree transformation preserving in-order sequence while changing shape.

## Detailed mechanics

### Ordering is identity inside a tree collection

For `TreeMap`, two keys are treated as the same map key when comparison returns zero. `equals` need not be consulted. For `TreeSet`, comparison zero means duplicate membership. Therefore a comparator inconsistent with equals can make the collection violate the general expectations of `Map` or `Set` equality, even while operating according to its own ordering.

This comparator collapses all people with the same age:

JAVA

```
Comparator<Person> byAgeOnly = Comparator.comparingInt(Person::age);
```

If distinct people of one age must coexist, add a stable tie-breaker:

JAVA

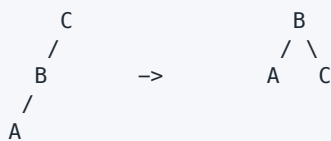
```
Comparator<Person> byAgeThenId = Comparator
    .comparingInt(Person::age)
    .thenComparing(Person::id);
```

The tie-breaker must reflect the intended identity and remain stable while stored.

### Search and insertion

Lookup starts at the root. A negative comparison descends left, positive descends right, and zero finds the key. Insertion follows the same path, attaches a new node, and restores balance. A rotation changes parent-child links without changing in-order traversal:

TEXT



Removal has extra cases. A leaf can be detached. A node with one child can be replaced by that child. A node with two children is logically replaced with its successor or predecessor, after which a simpler node is removed. Balanced implementations then repair color or height invariants.

**HotSpot note:** Current OpenJDK `TreeMap` uses a red-black tree and parent-linked entry nodes. Exact deletion strategy, color representation, and helper methods are version-sensitive.

## Navigational operations

**NavigableMap** offers `lowerEntry`, `floorEntry`, `ceilingEntry`, and `higherEntry`, along with key-returning forms. Entry-returning navigation methods produce snapshots of mappings rather than entries supporting `setValue`. `firstEntry` and `lastEntry` inspect endpoints; `pollFirstEntry` and `pollLastEntry` remove endpoints.

**NavigableSet** supplies equivalent element operations. These methods avoid the off-by-one logic that often appears in hand-written range code.

For key `20` in `{10, 20, 30}`:

### TEXT

```
lower(20)   = 10
floor(20)   = 20
ceiling(20) = 20
higher(20)  = 30
```

## Range and descending views

`subMap`, `headMap`, and `tailMap` expose backed views. Navigable overloads make endpoints explicit:

### JAVA

```
map.subMap(from, true, to, false) // [from, to)
```

Mutating a range view mutates the backing map. Inserting a key outside the range throws **`IllegalArgumentException`**. `descendingMap` and `descendingSet` are reverse-order views, not full copied reversals. Reversing again produces an equivalent view of the original order.

Iterators over ordinary tree maps, sets, and their views commonly detect unsupported structural mutation and fail fast. Detection is best-effort, not a concurrency guarantee; synchronize access or use a collection designed for concurrent navigation.

A stable snapshot requires copying. Copy into another **`TreeMap`** if the comparator and sorted navigation must be retained, or into a list if only the current ordered sequence is needed.

## Natural order, null, and heterogeneous keys

Natural ordering requires keys implementing mutually compatible `Comparable`. Mixing unrelated types can cause `ClassCastException`. Natural-order `TreeMap` rejects null keys because they cannot be compared. A custom comparator could define null placement, although null keys usually make domain modeling weaker.

Construct comparator chains carefully. `Comparator.comparing` can accept a key comparator, including `nullsFirst` or `nullsLast`. Do not implement numeric comparison with subtraction:

JAVA

```
// Wrong: overflow can reverse the sign.
(a, b) -> a.score() - b.score()

// Correct:
Comparator.comparingInt(Player::score)
```

## TreeSet relationship

A typical `TreeSet` is backed by a `NavigableMap` and stores elements as keys with a marker value. Its comparator, range behavior, navigation, and logarithmic costs follow the ordered map. The set API exposes only elements, preserving uniqueness under comparison.

**HotSpot note:** Backing `TreeSet` with a `TreeMap` is the familiar OpenJDK design, not a requirement that every Java implementation use the same fields.

## TRACE IT | Worked Java example

The following index finds the configuration effective at a given time:

JAVA (continued 1/2)

```
import java.time.Instant;
import java.util.NavigableMap;
import java.util.TreeMap;

record Config(String revision, int timeoutMillis) {
    public Config {
        if (timeoutMillis <= 0) {
            throw new IllegalArgumentException("timeout must be positive");
        }
    }
}

final class ConfigTimeline {
    private final NavigableMap<Instant, Config> byStart = new TreeMap<>();

    void publish(Instant startsAt, Config config) {
        if (startsAt == null || config == null) {
            throw new IllegalArgumentException("arguments must be non-null");
        }
        Config previous = byStart.putIfAbsent(startsAt, config);
    }
}
```

## JAVA (continued 2/2)

```

        if (previous != null) {
            throw new IllegalStateException("a revision already starts at " + startsAt);
        }
    }

    Config effectiveAt(Instant instant) {
        var entry = byStart.floorEntry(instant);
        if (entry == null) {
            throw new IllegalStateException("no configuration yet");
        }
        return entry.getValue();
    }

    NavigableMap<Instant, Config> changes(
        Instant fromInclusive, Instant toExclusive) {
        return java.util.Collections.unmodifiableNavigableMap(
            new TreeMap<>(byStart.subMap(
                fromInclusive, true, toExclusive, false)));
    }
}

```

The copy inside `changes` is deliberate. Returning only an unmodifiable wrapper around `subMap` would still expose later updates from the backing timeline. Here callers receive a membership snapshot that retains sorted navigation.

## TRACE IT | Execution or memory walkthrough

Suppose revisions start at 09:00, 12:00, and 18:00. Searching at 15:30 asks for `floorEntry(15:30)`. The tree follows comparisons until it either finds the key or reaches a missing child. During descent it remembers the best key seen that is below the target. The result is the 12:00 revision.

For a possible balanced shape:

## TEXT

```

      12:00(B)
     /      \
  09:00(R)  18:00(R)

```

Adding 20:00 may initially attach under 18:00. If red-black rules are violated, recoloring or rotations restore them while preserving this in-order sequence:

## TEXT

```

09:00, 12:00, 18:00, 20:00

```

The exact resulting shape and colors are not API-visible. Code should observe only ordering and mappings.

Each map entry typically carries key, value, left, right, parent, and balancing metadata. That is more per-entry memory than many hash-table entries and much more than flat arrays. A copied range allocates new tree entries but shares immutable `Instant` and `Config` references.

## Complexity and performance

For  $n$  entries in a balanced ordered tree:

| Operation                             | Time                                         | Additional space                                |
|---------------------------------------|----------------------------------------------|-------------------------------------------------|
| get, put, remove                      | $O(\log n)$                                  | $O(1)$ excluding inserted node                  |
| floor/ceiling/lower/higher            | $O(\log n)$                                  | $O(1)$                                          |
| first/last                            | $O(\log n)$ typical/documentated basic bound | $O(1)$                                          |
| ordered traversal                     | $O(n)$                                       | iterator state usually $O(1)$ with parent links |
| range traversal returning $k$ entries | $O(\log n + k)$ conceptual                   | view $O(1)$ , copy $O(k)$ nodes                 |

Comparator cost multiplies tree depth. Comparing long strings or extracting expensive sort keys can dominate. Precompute immutable comparison fields when justified.

**HashMap** usually provides lower expected point-lookup cost and lower comparison overhead. **TreeMap** earns its cost when sorted traversal, bounded ranges, or neighbor queries are first-class. If data changes rarely and is read often, a sorted array or list plus binary search may offer better locality and memory use.

### WATCH OUT | Edge cases and common mistakes

- Supplying a comparator that is not transitive, leading to unpredictable placement and lookup.
- Returning zero for distinct keys that must coexist.
- Mutating a field used by comparison while an element is stored.
- Using subtraction in integer comparators and overflowing.
- Confusing **lower** with **floor** or **higher** with **ceiling**.
- Forgetting that range bounds can be inclusive or exclusive.
- Returning a mutable backed range to an untrusted caller.
- Assuming a descending view is an independent reversed copy.
- Expecting null keys to work with natural ordering.
- Comparing heterogeneous keys without a comparator capable of handling both.
- Using **TreeMap** solely to print deterministic test output when sorting once would be simpler.
- Assuming the precise red-black tree shape or choosing behavior based on it.

## Production engineering notes

Centralize comparators as named, tested constants. Test sign symmetry, transitivity, zero behavior, and tie-breakers with generated data. Use stable immutable identifiers as final tie-breakers. When comparator equality intentionally differs from object equality, document that the tree collection defines uniqueness by ordering.

Range views are excellent within a short operation because they avoid copies. At service boundaries, prefer snapshots unless live coupling is a deliberate API feature. Treat `pollFirstEntry` and `pollLastEntry` as mutations and protect shared structures with an appropriate concurrency design; `TreeMap` is not thread-safe.

If a time index receives duplicate timestamps, decide whether one mapping, a list per timestamp, or a composite key is correct. Do not let an accidental comparator-zero replacement make that business decision. For very high read concurrency, consider publishing immutable sorted snapshots. For write-heavy concurrent navigation, evaluate purpose-built concurrent structures such as `ConcurrentSkipListMap` and its weaker snapshot semantics.

Measure retained memory and comparator CPU. Ordered indexes can duplicate data already held elsewhere. Decide which structure owns values, and remove stale entries consistently to avoid unbounded retention.

### TRY IT | Interview questions and model answers

**How does `TreeMap` differ from `HashMap`?**

`TreeMap` maintains keys in comparator or natural order and supports range and neighbor queries in  $O(\log n)$ . `HashMap` provides expected  $O(1)$  point operations but no ordering contract. The right choice follows required semantics.

**What happens when a comparator returns zero?**

The tree treats the keys as the same key or set element. A map insertion replaces the value for that ordering-equivalent key. Therefore tie-breakers are necessary when distinct objects must coexist.

**Why are tree operations logarithmic?**

A balancing invariant keeps height  $O(\log n)$ . Each search follows one root-to-leaf path, and rebalancing uses a bounded amount of local work per level.

**What is the difference between `floorKey` and `lowerKey`?**

`floorKey(x)` may return `x` itself and finds the greatest key  $\leq x$ . `lowerKey(x)` requires a strictly smaller key.

**Is `subMap` a copy?**

No. It is a backed, bounded view. Changes are reflected in the owner, and out-of-range insertions are rejected. Copy it when independent lifetime is needed.

**Must `TreeMap` be a red-black tree?**



No. The public contract promises behavior and complexity, not a specific balancing algorithm. Red-black structure describes current common OpenJDK implementation.

### TRY IT | Exercises

- 1 Design a comparator for orders sorted by descending priority, then creation time, then immutable ID. Explain why every tie-breaker is needed.
- 2 For keys `{10, 20, 30}`, compute lower, floor, ceiling, and higher results for targets 5, 20, 25, and 35.
- 3 Implement an interval lookup where each key is a range start. State what invariant is needed to avoid overlapping ranges.
- 4 Demonstrate how a comparator by string length causes `TreeSet` to collapse distinct same-length strings. Repair it.
- 5 Compare a sorted `ArrayList` plus binary search with `TreeMap` for one bulk build followed by one million reads.
- 6 Return an immutable snapshot of a descending range while preserving its comparator.

### RECAP | Chapter summary

Navigable tree collections maintain elements under a total ordering and support point, range, endpoint, and neighbor operations. Balanced-tree invariants keep height logarithmic; OpenJDK commonly uses red-black trees, but this is not the API contract. Comparison defines key identity inside the tree, so consistency, tie-breakers, and key stability are correctness requirements. Range and descending collections are backed views, making ownership choices as important as algorithmic complexity.

### TRY IT | Revision checklist

- ☐ I can state binary-search-tree and red-black-tree invariants.
- ☐ I understand that comparison zero defines uniqueness in tree collections.
- ☐ I can build safe comparator chains without subtraction overflow.
- ☐ I know lower, floor, ceiling, and higher semantics.
- ☐ I can use inclusive and exclusive range views correctly.
- ☐ I distinguish backed views from sorted snapshots.
- ☐ I can compare tree maps with hash maps and sorted arrays.
- ☐ I label red-black representation details as version-sensitive.

## SDE-2 CORE CHAPTER

# Chapter 6 - Queues, Deques, PriorityQueue, and Heaps

## Learning objectives

By the end of this chapter, you should be able to:

- select FIFO queues, double-ended queues, and priority queues by removal policy;
- use exception-throwing and special-value queue methods correctly;
- derive circular-buffer and binary-heap invariants;
- analyze enqueue, dequeue, heap maintenance, and iteration costs;
- avoid assumptions about priority-queue traversal, stability, and thread safety; and
- apply queues safely to scheduling, buffering, breadth-first search, and top-k problems.

### WHY IT WORKS | Why this matters at SDE-2

Queues define work order. They appear in request buffering, retries, graph traversal, schedulers, rate-limited pipelines, and event loops. Choosing the wrong removal policy can violate fairness or priority rules even when all operations compile.

Interviewers expect more than "a heap gives  $O(\log n)$ ." You should know which operation is logarithmic, why peeking is constant, why heap iteration is not sorted, and why finding an arbitrary element is linear. In production, you must also address capacity, overload, thread ownership, and tie-breaking. An unbounded in-memory queue can convert downstream slowness into an out-of-memory incident.

### WHY IT WORKS | First-principles model

A queue separates insertion policy from removal policy:

- A FIFO queue removes the oldest eligible element.
- A deque permits insertion and removal at both ends.
- A priority queue removes the least or greatest element under an ordering, not necessarily the oldest.

An array deque can use a circular logical sequence over a physical array:

## TEXT

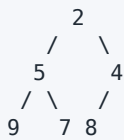
```
physical slots: [D, E, empty, empty, A, B, C]
logical order:  A, B, C, D, E
head index:     4
tail index:     2   (next insertion position)
```

Indices wrap around. The invariant is that logical elements occupy the circular range from head to tail according to the representation's empty/full convention.

A binary min-heap uses a complete binary tree whose parent is no greater than either child:

## TEXT

```
array: [2, 5, 4, 9, 7, 8]
```



For zero-based index  $i$ , children are commonly  $2i + 1$  and  $2i + 2$ ; parent is  $(i - 1) / 2$ . Completeness enables array storage without node links.

**Specification boundary:** Queue, Deque, and PriorityQueue define behavior. PriorityQueue documents heap-like complexity but does not promise a particular arity, array layout, growth policy, or stable ordering among equal-priority elements.

## Core terminology

- **Head:** Element selected for the next removal or inspection.
- **Tail:** End normally used for FIFO insertion.
- **FIFO:** First in, first out.
- **LIFO:** Last in, first out, or stack order.
- **Bounded queue:** Queue with a maximum capacity.
- **Backpressure:** Mechanism that slows, rejects, or redirects producers when consumers cannot keep up.
- **Heap:** Complete tree satisfying a local parent-child ordering invariant.
- **Sift up:** Move an inserted heap element toward the root until the invariant holds.
- **Sift down:** Move a displaced root toward leaves until the invariant holds.
- **Stable ordering:** Equal-priority elements retain their original relative order.
- **Tie-breaker:** Secondary comparison field that makes priority deterministic.

## Detailed mechanics

### Queue method pairs

The queue API offers two styles:

| Intent       | Throws on failure      | Returns special value                            |
|--------------|------------------------|--------------------------------------------------|
| insert       | <code>add(e)</code>    | <code>offer(e)</code> returns <code>false</code> |
| remove head  | <code>remove()</code>  | <code>poll()</code> returns <code>null</code>    |
| inspect head | <code>element()</code> | <code>peek()</code> returns <code>null</code>    |

For capacity-restricted queues, `offer` is usually the clearer insertion method because full capacity is an expected condition. `poll` and `peek` use `null` to mean empty, which is one reason queue implementations commonly reject `null` elements.

### Deque method families

Deque generalizes both ends. It provides `addFirst/offerFirst`, `addLast/offerLast`, `removeFirst/pollFirst`, and corresponding last methods. The queue aliases generally target the last for insertion and first for removal. Stack usage should prefer `push`, `pop`, and `peek` on a deque rather than the legacy `Stack` class.

`ArrayDeque` is usually the default general-purpose deque. It avoids a node allocation per element and offers good locality. It rejects `null`. `LinkedList` also implements `Deque`, but its per-node overhead and locality are usually worse.

**HotSpot note:** Current OpenJDK `ArrayDeque` uses a resizable circular array and head/tail indexes. Exact array-length conventions, growth increments, and wrap logic have changed across releases and are not public contracts.

### PriorityQueue heap mechanics

In a min-priority queue, the root is the minimum under natural ordering or a supplied comparator. Insertion appends at the next array position, then sifts upward while smaller than its parent. Removal saves the root, moves the last element to the root, decreases size, and sifts down by exchanging with the smaller child.

The heap invariant is local. It guarantees every parent is no greater than its children, which implies the root is globally minimal. It does not imply siblings or array positions are globally sorted. Thus iteration order is unspecified rather than priority order.

Ordinary deque and priority-queue iterators commonly fail fast after detected structural modification. This is diagnostic, best-effort behavior, not thread coordination. Iterator traversal is not a snapshot and still does not expose priority order.

For max-priority behavior, reverse the comparator. Be careful with arithmetic comparators; use `comparingInt`, `comparingLong`, or safe comparison methods instead of subtraction.

## Equal priorities and mutation

`PriorityQueue` does not guarantee FIFO order for equal elements. Add a monotonically increasing sequence as a final comparator field when stable tie-breaking is required. Consider sequence overflow and lifecycle; a `long` is ample for many process lifetimes but still deserves an explicit policy.

Never mutate an enqueued element's priority fields. The queue does not automatically locate and reposition it, so the heap can become logically invalid. Remove and reinsert the element, or enqueue immutable task descriptors and keep changing state elsewhere.

## Arbitrary operations and bulk heap construction

`contains` and `remove(Object)` generally scan the backing representation, costing  $O(n)$ , because the heap orders only by priority and has no key index. After an arbitrary removal is found, restoring the heap costs  $O(\log n)$ .

Building a heap by repeated insertion costs  $O(n \log n)$ . Bottom-up heap construction can heapify an array in  $O(n)$  because most nodes are near leaves and move only a short distance. A `PriorityQueue` constructor from a suitable collection may take advantage of bulk heapification; exact constructor paths remain implementation-specific.

## Bounded and concurrent queues

General-purpose `ArrayDeque` and `PriorityQueue` are unbounded in the API sense and not thread-safe. Capacity-aware and blocking behavior lives in concurrent queue types. A bounded blocking queue can wait, time out, or reject when full. A concurrent priority queue provides thread-safe priority access but still does not make a multi-step workflow atomic.

Queue selection must include who produces, who consumes, whether operations can block, and what happens under overload. Those are system semantics, not merely collection details.

## TRACE IT | Worked Java example

This retry scheduler orders tasks by due time and preserves insertion order among equal due times:

JAVA

```
import java.time.Instant;
import java.util.Comparator;
import java.util.PriorityQueue;

record Retry(long sequence, Instant dueAt, String jobId, int attempt) {}

final class RetrySchedule {
    private static final Comparator<Retry> ORDER = Comparator
        .comparing(Retry::dueAt)
        .thenComparingLong(Retry::sequence);

    private final PriorityQueue<Retry> queue = new PriorityQueue<>(ORDER);
    private long nextSequence;

    void schedule(Instant dueAt, String jobId, int attempt) {
        if (dueAt == null || jobId == null || attempt < 1) {
            throw new IllegalArgumentException("invalid retry");
        }
        queue.offer(new Retry(nextSequence++, dueAt, jobId, attempt));
    }

    Retry pollReady(Instant now) {
        Retry next = queue.peek();
        if (next == null || next.dueAt().isAfter(now)) {
            return null;
        }
        return queue.poll();
    }

    int pending() {
        return queue.size();
    }
}
```

This class assumes single-threaded ownership. A lock would be needed if multiple threads share it; using a thread-safe queue alone would not make the `peek` followed by conditional `poll` atomic. A production retry service also persists tasks, caps capacity, supports cancellation through an index, and handles sequence lifecycle.

## TRACE IT | Execution or memory walkthrough

Suppose the comparator sees tasks (12:00, seq 0), (12:10, seq 1), and (12:05, seq 2).

TEXT

```
insert 12:00/0: [12:00/0]
insert 12:10/1: [12:00/0, 12:10/1]
insert 12:05/2: append then compare with root
                [12:00/0, 12:10/1, 12:05/2]
```

The array is a valid heap even though positions 1 and 2 are not sorted with each other. Polling removes 12:00/0, moves 12:05/2 to the root, and sifts down:

#### TEXT

```
before removal: [12:00/0, 12:10/1, 12:05/2]
move last:      [12:05/2, 12:10/1]
result:         12:00/0
```

If another task due at 12:05 has sequence 3, sequence 2 sorts first. The tie-breaker turns an unspecified equal-priority outcome into a domain guarantee.

The priority queue stores references in an array and **Retry** record objects separately. Removed slots are cleared by normal implementations so they do not retain tasks. Exact capacity may exceed size. A deque similarly keeps spare slots to make end operations cheap.

## Complexity and performance

| Operation                   | ArrayDeque       | PriorityQueue                      |
|-----------------------------|------------------|------------------------------------|
| add/remove at supported end | amortized $O(1)$ | add $O(\log n)$ , poll $O(\log n)$ |
| peek head                   | $O(1)$           | $O(1)$                             |
| remove arbitrary object     | $O(n)$           | $O(n)$ search plus repair          |
| contains                    | $O(n)$           | $O(n)$                             |
| iterate all                 | $O(n)$           | $O(n)$ , not sorted                |
| copy then ordered drain     | $O(n)$ copy      | $O(n \log n)$ drain                |
| bottom-up heap construction | not applicable   | $O(n)$ algorithmically             |

Deque resize is occasionally  $O(n)$ , giving amortized constant end operations under geometric growth. Heap operations follow height  $O(\log n)$  because a complete binary tree with  $n$  nodes has logarithmic height.

For top-k selection from  $n$  values, maintain a heap of size  $k$ :  $O(n \log k)$  time and  $O(k)$  space. Sorting everything costs  $O(n \log n)$  and  $O(n)$  input storage or sorting space depending on representation. If  $k$  is close to  $n$ , constants and required output order can make sorting preferable.

## WATCH OUT | Edge cases and common mistakes

- Assuming iteration over `PriorityQueue` returns sorted order.
- Assuming equal-priority tasks are stable without a tie-breaker.
- Mutating an element's comparator fields while it is enqueued.
- Calling `remove(Object)` or `contains` frequently and expecting logarithmic behavior.
- Using null as a legitimate queue element when `poll` uses null for emptiness.
- Choosing exception-throwing `add` when full capacity is expected control flow.
- Using a queue that grows without bound under slower consumers.
- Sharing `ArrayDeque` or `PriorityQueue` across threads without protection.
- Separating `peek` and `poll` in concurrent code without one atomic policy.
- Using `PriorityQueue` for scheduling but ignoring wall-clock changes, durability, or wakeup coordination.
- Implementing a max heap by negating an integer and overflowing at `Integer.MIN_VALUE`.
- Assuming a heap is a binary search tree or that arbitrary search follows one branch.
- Forgetting stale duplicate entries in algorithms that reinsert improved priorities.

## Production engineering notes

Capacity is a reliability contract. Establish maximum depth, enqueue timeout or rejection behavior, retry/drop policy, and metrics for depth, age, throughput, and rejection. FIFO does not guarantee fairness if tasks have dramatically different service times. Priority scheduling can starve low-priority work; aging or quotas may be necessary.

Use immutable queue elements. When cancellation or priority updates are frequent, pair a heap with a map from ID to state, accept lazy deletion, or use a specialized indexed heap. Lazy deletion leaves stale entries until they reach the head, so bound and observe the overhead.

For breadth-first search, mark a node visited when enqueueing, not when dequeuing, to prevent duplicate queue growth. For stacks, prefer `ArrayDeque.push/pop` and never use null sentinels. For thread handoff, choose a concurrent or blocking queue whose ordering, capacity, and memory-consistency contract match the workflow.

Do not persist only an in-memory retry heap if process restart must not lose jobs. Keep durable source-of-truth state and rebuild the heap, or use an external scheduler. Collection choice solves in-process ordering, not distributed delivery guarantees.



## TRY IT | Interview questions and model answers

### Why is priority-queue iteration not sorted?

The heap invariant only orders each parent relative to its children. The backing array is a level-order heap representation, not a sorted sequence. Repeated polling, preferably from a copy, produces priority order.

### What are offer/poll/peek for?

They use special return values for expected failure: `offer` can report full capacity, while `poll` and `peek` return null for an empty queue. The paired methods `add/remove/element` throw exceptions.

### How does heap insertion work?

Append at the next complete-tree position, then sift upward while the new element precedes its parent. At most one root-to-leaf height is traversed, so cost is  $O(\log n)$ .

### Why is removing an arbitrary heap element $O(n)$ ?

The heap has no global search ordering for arbitrary values. It may scan all entries to locate the value, then needs only logarithmic repair.

### How do you make equal-priority scheduling deterministic?

Add a stable secondary comparator field, typically a monotonic sequence number or immutable ID, matching the desired business rule.

### Why prefer `ArrayDeque` over `Stack` or often `LinkedList`?

It directly implements deque and stack operations, avoids legacy synchronized `Vector` behavior, uses fewer objects than a linked list, and usually has better locality.

## TRY IT | Exercises

- 1 Dry-run heap insertion of 7, 3, 9, 1, 4 and then two polls. Draw the array after each step.
- 2 Use an `ArrayDeque` to implement breadth-first traversal and explain when each node is marked visited.
- 3 Find the largest five values in a stream using a min-heap of size five. State time and space bounds.
- 4 Design overload behavior for a bounded email-work queue. Include observability and retry policy.
- 5 Modify the retry scheduler to support lazy cancellation through a set of canceled IDs. Analyze stale-entry memory.
- 6 Demonstrate that printing or iterating a priority queue is not a sorted-output algorithm.

## RECAP | Chapter summary

Queues are defined by removal policy. Deques efficiently support both ends, while priority queues expose the minimum or maximum under an ordering. Circular arrays provide amortized constant deque operations; complete binary heaps provide constant-time peek and logarithmic insertion and head removal. Their local invariant does not sort iteration or accelerate arbitrary search. Production use adds bounded capacity, overload behavior, immutability, concurrency ownership, fairness, and durability.

## TRY IT | Revision checklist

- ☐ I know the exception and special-value queue method pairs.
- ☐ I can map every **Deque** operation to an end and use it as a stack.
- ☐ I can state circular-buffer and min-heap invariants.
- ☐ I can dry-run sift-up and sift-down.
- ☐ I do not assume priority-queue iteration or equal-priority stability.
- ☐ I know why arbitrary heap search and removal are linear.
- ☐ I can derive  $O(n \log k)$  top-k selection.
- ☐ I include capacity, backpressure, concurrency, fairness, and durability in production designs.

## SDE-2 CORE CHAPTER

# Chapter 7 - Comparable, Comparator, Sorting, and Selection

## Learning objectives

By the end of this chapter, you should be able to:

- define natural and external orderings with valid comparison contracts;
- compose null-safe, deterministic comparators without overflow;
- explain stable sorting, in-place sorting, and comparison lower bounds;
- choose among object sort, primitive sort, parallel sort, heap selection, and quickselect;
- use binary search only under its ordering precondition; and
- discuss current OpenJDK algorithms without confusing them with API guarantees.

### WHY IT WORKS | Why this matters at SDE-2

Sorting is often the boundary between raw data and useful output: ranked results, merge pipelines, pagination, reconciliation, and deduplication all depend on ordering. Comparison defects can be intermittent and severe. A non-transitive comparator may make sorting fail at runtime, corrupt a tree collection's model, or create unstable pagination.

At SDE-2, you should translate a product rule into a total order, handle ties explicitly, and select an algorithm based on whether all values or only the top *k* are needed. You should distinguish specification promises, such as stability for an object sort, from OpenJDK's current TimSort or primitive-array algorithms.

### WHY IT WORKS | First-principles model

An ordering comparator maps two values to a negative number, zero, or a positive number:

#### TEXT

```
compare(a, b) < 0  means a precedes b
compare(a, b) = 0  means a and b are ordering-equivalent
compare(a, b) > 0  means a follows b
```

The magnitude is irrelevant. A comparator must behave like a total order over the accepted domain. Important laws are:

- sign symmetry: `sign(compare(a,b)) == -sign(compare(b,a));`

- transitivity: if  $a > b$  and  $b > c$ , then  $a > c$ ;
- zero consistency: if  $a$  compares equal to  $b$ , comparisons of each against  $c$  have the same sign; and
- repeatability while participating fields remain unchanged.

Comparison sorting uses pairwise questions to distinguish possible permutations. There are  $n!$  permutations, so a decision tree requires height  $\Omega(\log(n!))$ , which is  $\Omega(n \log n)$ . General comparison sorting cannot asymptotically beat that bound. Algorithms using restricted keys, such as counting sort, operate under different assumptions.

**Specification boundary:** `Comparable` and `Comparator` define ordering contracts. Sorting APIs document properties such as stability and permitted mutation. Their exact algorithm, run detection, pivot selection, temporary storage, and thresholds can vary by JDK implementation and version.

## Core terminology

- **Natural order:** A type's canonical order implemented by `Comparable`.
- **External order:** A purpose-specific `Comparator` supplied by a client.
- **Stable sort:** Equal-order elements retain input relative order.
- **In-place:** Uses only bounded or logarithmic auxiliary storage under a stated model; common library documentation may use looser wording.
- **Adaptive sort:** Exploits existing order or runs in the input.
- **Total order:** Consistent ordering for every accepted pair.
- **Partial selection:** Find a rank or top subset without fully sorting.
- **Quickselect:** Partition-based expected linear-time selection.
- **Partition:** Rearrange elements around a pivot by comparison.
- **Tie-breaker:** Additional field distinguishing otherwise equal rankings.
- **Schwartzian transform/decorate-sort-undecorate:** Precompute expensive sort keys, sort decorated values, then extract originals.

## Detailed mechanics

### Comparable versus Comparator

Implement `Comparable<T>` when a type has one unsurprising natural order used broadly:

JAVA

```
record Version(int major, int minor, int patch)
    implements Comparable<Version> {
    @Override
    public int compareTo(Version other) {
        int result = Integer.compare(major, other.major);
        if (result != 0) return result;
        result = Integer.compare(minor, other.minor);
        if (result != 0) return result;
        return Integer.compare(patch, other.patch);
    }
}
```

Use comparators for alternative views: orders by creation time, priority, customer, or amount. A natural order becomes part of a public type's long-lived meaning, so do not add one merely for a single screen.

Natural ordering should usually be consistent with `equals`, especially when values enter sorted sets or maps. `BigDecimal` is a notable counterexample: values such as `1.0` and `1.00` compare as zero but are not equal because scale affects `equals`. A `TreeSet<BigDecimal>` can therefore have different uniqueness behavior from a `HashSet<BigDecimal>`.

### Comparator composition

Comparator factories make intent explicit:

JAVA

```
Comparator<Order> order = Comparator
    .comparingInt(Order::priority).reversed()
    .thenComparing(Order::createdAt)
    .thenComparing(Order::id);
```

Apply reversal carefully. Calling `reversed()` at the end reverses the entire chain. Calling it after the primary comparator reverses only that comparator before adding subsequent ascending tie-breakers.

Use primitive factories to avoid boxing in comparison: `comparingInt`, `comparingLong`, and `comparingDouble`. Use `Comparator.nullsFirst` or `nullsLast` only if null is valid domain data. Do not compare integers with `a - b`; overflow can violate the sign rule. Floats and doubles also require library comparison because NaN and signed zero need a consistent total order.

## Stability and tie-breaking

Stable sort preserves input order when comparator returns zero. This permits multi-pass sorting: stable-sort by a secondary key, then by primary key. A single comparator chain is usually clearer and performs fewer sorts.

Stability is not a substitute for a deterministic total presentation order. If input arrives from a hash map or concurrent source with unspecified encounter order, stable sorting equal ranks preserves an unspecified order. Add a unique immutable ID as a tie-breaker for reproducible pagination and tests.

## Library sorting APIs

`List.sort(comparator)` mutates the list and is specified as stable. `Collections.sort` delegates to the list's sorting facility in modern APIs. The list must support the required replacement operation; an unmodifiable list rejects sorting.

`Arrays.sort(Object[])` is stable. Primitive overloads are not required to be stable because primitive values have no separately observable identity when equal. Primitive sorts avoid boxing and are normally much more memory-efficient.

`Arrays.parallelSort` may use the common fork-join pool and temporary storage. Parallel overhead can outweigh gains for small arrays or contended services. Measure on the deployment shape, and remember that comparator work must be thread-safe and side-effect-free.

**HotSpot note:** Current OpenJDK releases commonly use TimSort-derived logic for object arrays and lists, dual-pivot quicksort and specialized paths for several primitive types, and parallel merge/sort strategies above thresholds. Algorithms and thresholds are version-sensitive.

## TimSort intuition and comparator failures

TimSort is adaptive: it discovers ascending or descending runs, extends short runs, and merges runs while maintaining stack invariants. Nearly sorted input can therefore be efficient. It needs temporary storage for merging.

Inconsistent comparators can trigger exceptions such as "comparison method violates its general contract" in some sort paths, but detection is not guaranteed. A sort that appears to complete does not prove comparator correctness.

## Selection instead of full sorting

If only the smallest or largest  $k$  items are needed:

- sort all:  $O(n \log n)$  time, straightforward, produces complete order;
- maintain a heap of size  $k$ :  $O(n \log k)$  time and  $O(k)$  extra space;
- quickselect: expected  $O(n)$  to place the  $k$ th rank, then optionally sort the selected  $k$ ; worst case  $O(n^2)$  without robust pivot strategy;
- counting/bucket techniques: potentially  $O(n + \text{range})$  when integer key range is small and bounded.

Quickselect partitions around a pivot. After partition, if the pivot lands at rank  $k$ , selection is complete; otherwise recurse or iterate only on the side containing rank  $k$ . It mutates the array unless implemented on a copy.

## Binary search

`Collections.binarySearch` and `Arrays.binarySearch` require data sorted according to the same ordering. A nonnegative result is a matching index. A negative result encodes insertion point  $-(\text{result}) - 1$ . When duplicates exist, no promise should be inferred about which equal occurrence is returned unless the API states one. Use explicit lower-bound and upper-bound searches for ranges of duplicates.

## TRACE IT | Worked Java example

This method returns the top  $k$  candidates while bounding intermediate storage:

JAVA (continued 1/2)

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

record Candidate(String id, int score, long completedAtEpochMillis) {}

final class Ranking {
    // Best first: higher score, earlier completion, lexicographically smaller ID.
    private static final Comparator<Candidate> BEST_FIRST = Comparator
        .comparingInt(Candidate::score).reversed()
        .thenComparingLong(Candidate::completedAtEpochMillis)
        .thenComparing(Candidate::id);

    static List<Candidate> topK(Collection<Candidate> input, int k) {
        if (k < 0) throw new IllegalArgumentException("k must be non-negative");
        if (k == 0) return List.of();
    }
}
```

JAVA (continued 2/2)

```
// Worst retained candidate is at the head.
PriorityQueue<Candidate> retained =
    new PriorityQueue<>(k, BEST_FIRST.reversed());

for (Candidate candidate : input) {
    if (candidate == null) throw new IllegalArgumentException("null candidate");
    if (retained.size() < k) {
        retained.offer(candidate);
    } else if (BEST_FIRST.compare(candidate, retained.peek()) < 0) {
        retained.poll();
        retained.offer(candidate);
    }
}

ArrayList<Candidate> result = new ArrayList<>(retained);
result.sort(BEST_FIRST);
return List.copyOf(result);
}
```

The ID tie-breaker gives a deterministic total ranking. If IDs are unique, no two distinct candidates compare as zero. The heap uses reverse order so its head is the worst candidate currently retained; a better incoming candidate replaces it.

### TRACE IT | Execution or memory walkthrough

For  $k = 3$  and candidates with scores 70/A, 90/B, 80/C, 85/D, and 80/E, assume times and IDs do not change the score comparisons:

- 1 Retain A, B, C. The heap head is A, the worst of those three.
- 2 D outranks A, so poll A and add D. Retained set is B, C, D; C is now worst.
- 3 E ties C on score. Completion time and then ID decide whether E is better. Replace only if the full comparator ranks E before C.
- 4 Copy the heap. Its iteration order is not ranked.
- 5 Sort the copy best-first and publish an unmodifiable result.

At most  $k$  candidate references reside in the heap, plus the final list of at most  $k$  references. Candidate records are shared rather than copied. Comparator calls extract primitive fields without boxing for score and time.

Quickselect would store the entire mutable input or a copy but could reduce asymptotic selection time. The heap supports one-pass input and is appropriate when the collection is streamed or  $k$  is small.



## Complexity and performance

For **topK**, each of  $n$  inputs performs constant comparison plus at most one heap replacement costing  $O(\log k)$ . Time is  $O(n \log k + k \log k)$  and auxiliary reference storage is  $O(k)$ . When  $k \geq n$ , the behavior approaches  $O(n \log n)$  and sorting all may be simpler and faster.

Common sorting bounds:

| Technique       | Time                                                       | Extra space               | Stable                                 | Best use                                    |
|-----------------|------------------------------------------------------------|---------------------------|----------------------------------------|---------------------------------------------|
| comparison sort | $O(n \log n)$<br>typical/guaranteed by<br>chosen algorithm | varies                    | API-dependent                          | full order                                  |
| insertion sort  | $O(n^2)$ , near $O(n)$ when<br>nearly sorted               | $O(1)$                    | yes when implemented<br>conventionally | tiny/nearly sorted<br>ranges                |
| merge sort      | $O(n \log n)$                                              | $O(n)$                    | yes                                    | stable predictable<br>sorting               |
| heap sort       | $O(n \log n)$                                              | $O(1)$ array<br>model     | no                                     | worst-case bound, low<br>extra space        |
| quicksort       | average $O(n \log n)$ , worst<br>$O(n^2)$                  | recursion/stack<br>varies | usually no                             | fast<br>primitive/in-place-style<br>sorting |
| heap top-k      | $O(n \log k)$                                              | $O(k)$                    | only with explicit tie<br>order        | small top subset                            |
| quickselect     | expected $O(n)$ , worst<br>$O(n^2)$                        | often $O(1)$<br>iterative | no                                     | one rank or unsorted<br>partition           |

Comparator cost can dominate  $n \log n$ . Avoid network calls, database access, mutable clocks, locale creation, or repeated expensive parsing inside comparison. Precompute keys when profiling justifies it.

## WATCH OUT | Edge cases and common mistakes

- Implementing compare with subtraction and overflowing.
- Omitting a stable tie-breaker for pagination or distributed merge results.
- Assuming stable sort makes unspecified input order deterministic.
- Using a comparator that changes based on mutable state, current time, or side effects.
- Mixing natural and external ordering between sort and binary search.
- Expecting binary search to return the first duplicate.
- Sorting an unmodifiable or fixed-capability list without understanding supported operations.
- Mutating comparator fields while objects reside in a sorted collection.
- Assuming primitive and object sort stability are identical.
- Parallelizing a small sort or using a non-thread-safe comparator with parallel sort.
- Fully sorting millions of items to return a tiny top-k result.
- Forgetting that `reversed()` placement can reverse an entire comparator chain.
- Treating current OpenJDK algorithm names or thresholds as portable guarantees.

## Production engineering notes

Define ordering once and reuse it across database queries, in-memory merge, API pagination, and tests. Any mismatch can cause duplicates or gaps between pages. Cursor pagination needs a unique final tie-breaker included in both ordering and cursor encoding.

Comparator functions should be pure, cheap, null-explicit, and tested with property-style checks for symmetry and transitivity. Normalize text before sorting if case, Unicode, or locale rules matter. Human-language collation is not equivalent to `String` code-unit order and may depend on locale/version; isolate it from identity ordering.

Avoid sorting shared mutable lists in place. Copy, sort, and publish when readers need snapshots. For large results, push sorting and limiting toward an indexed data source when possible. Enforce input caps for user-controlled sorts to prevent CPU and memory abuse.

Measure sequential versus parallel sorting in realistic service contention. A common pool is shared process capacity, not free compute. Primitive arrays avoid wrapper allocation and should be preferred for numerical kernels when the surrounding design permits them.

## TRY IT | Interview questions and model answers

### When should a class implement `Comparable`?

When it has one canonical, unsurprising natural order that is meaningful across uses. Purpose-specific orders belong in named comparators.

### What properties must a comparator satisfy?

It must provide consistent sign symmetry, transitivity, and zero behavior over its accepted domain. It should be repeatable while compared state is unchanged and preferably consistent with equals for collection use.

### What does stable sort mean?

Elements that compare as equal retain their original relative order. It does not create determinism if original encounter order is unspecified.

### How would you find the top 100 of ten million values?

Use a size-100 min-heap while scanning:  $O(n \log 100)$  time and  $O(100)$  memory, then sort the retained items for final output. Discuss database pushdown or distributed merge if data is external.

### Why can quickselect be faster than sorting?

It explores only the partition containing the target rank, giving expected linear work. It does not fully order the data and has quadratic worst cases without pivot safeguards.

### Can I rely on TimSort for every Java object sort?

Rely on the documented stability and behavior of the API, not an algorithm name. TimSort is a common current OpenJDK implementation and can change.

## TRY IT | Exercises

- 1 Write a comparator for nullable invoices ordered by status, descending amount, due date, and unique ID. State null policy.
- 2 Produce a three-value counterexample showing how a bad cyclic comparator violates transitivity.
- 3 Implement lower-bound binary search that returns the first index whose element is not less than a target.
- 4 Compare full sort, heap top-k, and quickselect for  $n = 1,000,000$  and  $k = 10$ , then for  $k = 900,000$ .
- 5 Repair a comparator written as  $(a, b) \rightarrow a.timestamp() > b.timestamp() ? 1 : -1$ .
- 6 Explain why stable sorting entries from a `HashMap` by value alone can still produce changing output.

## RECAP | Chapter summary

Ordering is a correctness contract before it is an algorithm. `Comparable` defines a canonical natural order; `Comparator` defines external orders that can be composed with safe tie-breakers. Stable full sorting solves complete ordering in  $O(n \log n)$ , while heaps and selection algorithms avoid unnecessary work for small subsets. Java APIs specify observable properties, whereas TimSort, primitive quicksort variants, and thresholds are current implementation choices. Production ordering must align across layers and remain deterministic, pure, and bounded.

## TRY IT | Revision checklist

- ☐ I can state comparator laws and consistency-with-equals concerns.
- ☐ I use safe primitive comparisons rather than subtraction.
- ☐ I understand reversal placement and comparator chaining.
- ☐ I distinguish stability from deterministic total ordering.
- ☐ I know object, primitive, and parallel sort trade-offs.
- ☐ I can choose full sort, heap top-k, or quickselect from requirements.
- ☐ I use binary search only with the same ordering used for sorting.
- ☐ I label OpenJDK sorting algorithms and thresholds as version-sensitive.

## SDE-2 CORE CHAPTER

# Chapter 8 - Streams, Collectors, Optional, and Spliterators

## Learning objectives

By the end of this chapter, you should be able to:

- explain lazy stream pipelines, encounter order, and single-use traversal;
- distinguish stateless, stateful, short-circuiting, intermediate, and terminal operations;
- write side-effect-free transformations and associative reductions;
- compose collectors and state their mutability, duplicate, null, and ordering behavior;
- use `Optional` at absence-return boundaries without turning it into a universal container;
- interpret spliterator characteristics and splitting; and
- evaluate parallel streams using correctness laws, source shape, workload, and deployment context.

### WHY IT WORKS | Why this matters at SDE-2

Streams compress collection processing into declarative pipelines, but concise syntax can hide multiple traversals, large buffers, unsafe side effects, and ambiguous merge behavior. At SDE-2, you should be able to review a pipeline as rigorously as a loop: identify its source, cardinality changes, ordering, terminal result, allocation, and parallel assumptions.

Collectors appear in grouping, aggregation, indexing, and reporting. `Optional` appears at repository and lookup boundaries. Spliterators connect collection data structures to sequential and parallel traversal. Understanding all three prevents accidental  $O(n^2)$  work, data races, and APIs that are harder to use than a simple null check or domain result type.

### WHY IT WORKS | First-principles model

A stream is not a container. It is a one-shot description of a traversal and transformation. A pipeline has:

TEXT

```
source -> intermediate operations -> terminal operation
```

Intermediate operations return another stream and are generally lazy. The terminal operation pulls elements through the pipeline. A fused sequential pipeline can process one element through several stages before requesting the next.

## TEXT

```
input:  [1, 2, 3, 4, 5]
filter even -> map square -> findFirst

1 rejected
2 accepted -> 4 -> terminal completes
3, 4, 5 may never be requested
```

A collector is a mutable-reduction recipe with four conceptual functions:

- 1 supplier creates an accumulator;
- 2 accumulator incorporates one input;
- 3 combiner merges partial accumulators; and
- 4 finisher converts the accumulator to the result, unless identity finishing applies.

A spliterator is a traversal cursor that can also partition remaining work. Its characteristics describe properties a caller may exploit.

**Specification boundary:** Stream semantics define laziness, operation contracts, encounter-order rules, reduction requirements, and single use. They do not guarantee stage fusion strategy, thread count, partition sizes, common-pool scheduling, vectorization, or that parallel execution will be faster.

## Core terminology

- **Pipeline:** Source, intermediate operations, and one terminal operation.
- **Lazy:** Work is deferred until demanded by a terminal operation.
- **Encounter order:** Order in which a stream logically presents elements when its source or operations define one.
- **Stateless operation:** Result for one element does not depend on previously seen elements, such as `map`.
- **Stateful operation:** May buffer or track elements, such as `sorted` or `distinct`.
- **Short-circuiting:** May complete without processing all input, such as `findFirst` or `limit`.
- **Non-interference:** Pipeline behavior does not modify or depend on unsafe mutation of its source during execution.
- **Associative:** Grouping operands differently produces an equivalent result.
- **Mutable reduction:** Accumulate into a mutable result container, as with `collect`.
- **Downstream collector:** Collector applied to values within each group or partition.
- **Characteristic:** Spliterator or collector property clients can use for execution decisions.
- **Primitive stream:** `IntStream`, `LongStream`, or `DoubleStream`, avoiding wrapper elements for many operations.

## Detailed mechanics

### Laziness, fusion, and single use

`filter`, `map`, `flatMap`, `peek`, `distinct`, `sorted`, `limit`, and `skip` are intermediate. `forEach`, `reduce`, `collect`, `count`, `findFirst`, and matching operations are terminal. Without a terminal operation, a pipeline performs no traversal.

A stream can be consumed once. Reusing it after a terminal operation throws `IllegalStateException`. Store a source or a `Supplier<Stream<T>>` when repeated traversal is intended.

Laziness enables fusion and short-circuiting, but stateful operations can create barriers. `sorted` usually needs all relevant input before emitting the first sorted result. `distinct` tracks seen values. On ordered parallel pipelines, `limit`, `skip`, and `distinct` can require coordination and buffering.

### Operation classes and ordering

`map` preserves cardinality one-for-one; `filter` can reduce it; `flatMap` maps one input to zero or more outputs and closes each nested stream after use. `flatMapMulti`, available in modern Java, can emit zero or more outputs without constructing a stream per source item.

Ordered sources include lists and sorted collections. Hash-based collections generally do not promise encounter order. `forEachOrdered` respects encounter order when one exists, even in a parallel pipeline, at a coordination cost. `findFirst` is order-sensitive; `findAny` permits more freedom and can be useful in parallel.

Calling `unordered()` does not randomly shuffle data. It removes the obligation to preserve encounter order, allowing optimizations for operations whose result does not require it.

### Side effects and non-interference

Behavioral parameters should be stateless and non-interfering:

JAVA

```
// Wrong for parallel use and difficult even sequentially.
List<String> output = new ArrayList<>();
input.parallelStream()
    .filter(this::valid)
    .forEach(output::add);

// Correct mutable reduction.
List<String> output = input.parallelStream()
    .filter(this::valid)
    .toList();
```

Concurrent mutation of the source during traversal may throw, produce weakly consistent observations for a concurrent source, or otherwise follow that source's contract. Streams do not add isolation.

`peek` exists mainly for debugging and carefully controlled observation. It may not execute for every conceptual element because short-circuiting and implementation optimizations can avoid traversal. Do not put required business effects in `peek`.

## Reduction laws

For `reduce(identity, accumulator, combiner)`, the identity must be neutral, the reduction must be associative, and accumulator/combiner behavior must be compatible. Integer addition satisfies this ignoring overflow as Java-defined wraparound grouping effects for fixed values; subtraction does not:

TEXT

```
(10 - 3) - 2 = 5  
10 - (3 - 2) = 9
```

A sequential left fold with subtraction may look predictable, but parallel partitioning changes grouping. Floating-point addition is mathematically associative but not bitwise associative due to rounding, so parallel sums can differ slightly.

Do not mutate the identity object in the three-argument `reduce`. Multiple partitions may share assumptions incompatible with mutable state. Use `collect` for mutable containers.

## Collector composition

Common collectors include:

- `toList`, `toSet`, and `toCollection`;
- `joining` for character sequences;
- `counting`, `summingInt`, `averagingLong`, and `summarizingDouble`;
- `groupingBy` and `groupingByConcurrent`;
- `partitioningBy` for exactly two Boolean groups;
- `mapping`, `filtering`, and `flatMap` downstream;
- `collectingAndThen` for a finishing transformation; and
- `teeing` to run two downstream aggregations and merge their results.

When collecting to a map, duplicate keys need policy:

JAVA

```
Map<String, User> byEmail = users.stream().collect(  
    java.util.stream.Collectors.toMap(  
        User::email,  
        java.util.function.Function.identity(),  
        (left, right) -> chooseNewer(left, right),  
        java.util.LinkedHashMap::new));
```

Without a merge function, duplicate keys throw `IllegalStateException`. That can be the right invariant check. The map supplier determines a concrete result representation when an overload accepts it.

`Collectors.toList()` does not promise mutability, unmodifiability, or a concrete list type.

`Stream.toList()` returns an unmodifiable list and may have different null behavior from `List.copyOf`;



do not substitute methods solely because names look similar. `Collectors.toUnmodifiableList()` rejects null elements.

Collector characteristics include `CONCURRENT`, `UNORDERED`, and `IDENTITY_FINISH`. `CONCURRENT` does not mean every use updates one shared result concurrently; source ordering and collector characteristics influence the strategy. Custom collectors must make accumulator, combiner, and finisher laws correct before seeking performance.

## Optional as an absence result

`Optional<T>` models either one non-null value or absence. It works well as a return type for lookups:

JAVA

```
Optional<User> findById(UserId id)
```

Useful methods include `map`, `flatMap`, `filter`, `or`, `orElseGet`, `ifPresentOrElse`, and `stream.map` wraps a non-null mapping result and yields empty for null. `flatMap` is for functions already returning `Optional`.

`orElse(fallback())` evaluates `fallback()` eagerly. `orElseGet(this::fallback)` invokes it only when empty. `orElseThrow` is appropriate when absence violates the current layer's precondition.

Avoid `Optional` fields, collection elements, and method parameters by default. They often add a second wrapper state without improving the domain. A list already represents zero or more values; return an empty list instead of `Optional<List<T>>` unless absence and empty are genuinely different. `Optional` is value-based: do not synchronize on instances or rely on identity.

Specialized `OptionalInt`, `OptionalLong`, and `OptionalDouble` avoid boxing. `Optional` is not a substitute for a rich failure result containing reason, retryability, or validation errors.

## Spliterator mechanics

A spliterator supports `tryAdvance(action)` for one element, `forEachRemaining`, `trySplit`, `estimateSize`, and `characteristics`. Common characteristics are:

- **ORDERED**: defined encounter order;
- **DISTINCT**: elements are distinct under relevant equality;
- **SORTED**: encounter order is sorted and a comparator may be available;
- **SIZED**: exact remaining size is known;
- **SUBSIZED**: split children also report exact sizes;
- **NONNULL**: source guarantees non-null elements;
- **IMMUTABLE**: source cannot be structurally modified;
- **CONCURRENT**: source supports concurrent structural modification under its contract.

`trySplit` returns a prefix or partition and leaves remaining work in the original. Balanced, cheap splitting enables useful parallelism. Array and range sources split well. Pointer-chasing, unknown-size, or I/O-backed sources may split poorly.

Characteristics are promises, not hints. A custom spliterator that falsely reports **`SORTED`**, **`SIZED`**, or **`DISTINCT`** can cause wrong optimizations or contract violations. Late-binding and fail-fast behavior depend on the source implementation.

**HotSpot note:** OpenJDK stream execution uses internal pipeline stages, fork-join tasks for many parallel pipelines, and heuristics for target partition sizes. These details and optimization shortcuts are version-sensitive.

### Parallel stream decision model

Parallel streams are most plausible when input is large, CPU work per element is substantial, operations are associative and independent, splitting is balanced, and no blocking or shared mutable state is involved. They are often poor for small inputs, ordered stateful stages, blocking I/O, request-thread latency isolation, or environments where the common pool is already busy.

Measure end-to-end behavior under concurrent service load. A microbenchmark showing one pipeline speedup does not prove better tail latency or capacity in a server.

## TRACE IT | Worked Java example

The following aggregation produces immutable per-region statistics from completed orders:

JAVA

```
import java.math.BigDecimal;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

enum Status { PENDING, COMPLETED, CANCELED }
record Order(String id, String region, Status status, BigDecimal amount) {}
record RegionSummary(long count, BigDecimal total, List<String> orderIds) {}

final class OrderReport {
    static Map<String, RegionSummary> summarize(List<Order> orders) {
        return orders.stream()
            .filter(order -> order.status() == Status.COMPLETED)
            .collect(Collectors.groupingBy(
                Order::region,
                java.util.TreeMap::new,
                Collectors.collectingAndThen(
                    Collectors.toList(),
                    OrderReport::summarizeGroup)));
    }

    private static RegionSummary summarizeGroup(List<Order> group) {
        BigDecimal total = group.stream()
            .map(Order::amount)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

        List<String> ids = group.stream()
            .sorted(Comparator.comparing(Order::id))
            .map(Order::id)
            .toList();

        return new RegionSummary(group.size(), total, ids);
    }
}
```

The outer `TreeMap` guarantees region order. Each list is an intermediate mutable accumulator local to collection, and the finisher converts it to an immutable summary. `BigDecimal.add` returns a new value, so there is no shared mutable numeric accumulator.

For a very large group, retaining its entire list may be unnecessary. A custom accumulator could track count, total, and IDs directly, but its combiner must merge all fields correctly. If sorted IDs are mandatory, memory proportional to group cardinality remains necessary unless output can stream to another sorted source.

## TRACE IT | Execution or memory walkthrough

Given completed orders `east/e2/$5`, `west/w1/$7`, and `east/e1/$3`, plus one canceled east order:

- 1 `filter` rejects the canceled order.
- 2 `groupingBy` asks the `TreeMap` for an accumulator list for each region.
- 3 East's list receives `e2` and `e1`; west's receives `w1`.
- 4 The downstream finisher visits each group. East total becomes  $0 + 5 + 3 = 8$ ; IDs sort to `[e1, e2]`.
- 5 The final map is sorted by region, but it remains mutable because `groupingBy` with `TreeMap::new` returns that mutable map. If an unmodifiable outer map is required, wrap the complete collector with `collectingAndThen` and an order-preserving unmodifiable snapshot strategy.

In a parallel execution, partitions can build separate maps and lists, then combiners merge them. Encounter order for downstream lists follows the collector and source contracts; code must not assume one shared accumulator. The example remains mathematically correct because filters and mappings are pure and `BigDecimal` addition is associative in value for these exact decimal inputs, subject to any chosen math context.

Memory includes group lists containing references to orders, final ID lists containing references to existing strings, summary records, and tree map entries. Streams themselves add pipeline objects and possibly tasks/buffers, but they do not automatically duplicate every input.

## Complexity and performance

Let  $n$  be input size,  $g$  number of regions, and  $m_i$  each group size. Filtering and grouping perform  $O(n)$  element work plus `TreeMap` group lookup  $O(\log g)$  per accepted element, so  $O(n \log g)$  under this chosen map. Summing is  $O(n)$ . Sorting IDs costs:

TEXT

sum over groups of  $O(m_i \log m_i)$ , bounded by  $O(n \log n)$

Space is  $O(n + g)$  because group lists and final ID lists retain per-order references during finishing. A one-pass custom collector could reduce temporary duplication but not the final list storage.

Pipeline complexity composes by stages. `map` and `filter` are normally linear; `sorted` is  $O(n \log n)$ ; `distinct` is expected  $O(n)$  with hashing but depends on source/order and equality cost; `limit(k)` can be  $O(k)$  sequentially with a favorable source but may coordinate more work in parallel.

Boxing matters in numerical pipelines. `stream.mapToLong(...).sum()` avoids one wrapper per mapped value. It does not fix an expensive source, I/O, or algorithm. Benchmark with JMH-style discipline and realistic cardinality before replacing a clear loop.

## WATCH OUT | Edge cases and common mistakes

- Reusing a stream after a terminal operation.
- Mutating a non-thread-safe collection from `parallel().forEach`.
- Using `peek` for required persistence or audit effects.
- Supplying a non-associative reduction and observing parallel differences.
- Mutating a shared identity in `reduce` instead of using `collect`.
- Forgetting duplicate-key behavior in `toMap`.
- Assuming collector-produced collection types, mutability, null policy, or order without a contract.
- Calling `orElse(expensive())` when lazy `orElseGet` is intended.
- Calling `optional.get()` without proving presence, recreating a less clear null check.
- Using `Optional<List<T>>` when empty list already expresses no results.
- Flat-mapping infinite streams without a termination argument.
- Placing `sorted` before a selective `filter` and sorting unnecessary elements.
- Parallelizing blocking I/O on the common pool.
- Reporting false characteristics in a custom spliterator.
- Depending on fail-fast traversal to make source mutation safe.
- Assuming `unordered()` shuffles or that `parallel` automatically ignores encounter order.

## Production engineering notes

Prefer pipelines whose stages read like a data-flow specification and whose cardinality remains bounded. Break a complex pipeline into named methods or intermediate results when it improves observability and debugging. A loop is often better when error handling, early exits, multiple stateful outputs, or checked resource lifetimes dominate.

Keep lambdas pure. Move database calls and remote requests outside element-wise stream operations; otherwise a compact line can hide  $N+1$  I/O. Batch external work explicitly and use concurrency mechanisms with controlled executors, timeouts, and backpressure.

Define output map/list order and mutability. Use explicit collection suppliers when representation matters. Treat `groupingBy` on unbounded cardinality as a memory risk. Validate keys, cap results, and avoid `distinct` on attacker-controlled unbounded streams.

Use `Optional` for an immediate maybe-one return, then unwrap or transform near that boundary. Domain-specific sealed results are clearer when absence has multiple causes. Avoid `Optional` in serialization entities unless the serializer and schema deliberately support it.

For parallel pipelines, validate algebraic correctness sequentially first. Then benchmark using the target JDK, CPU quota, container limits, and concurrent traffic. Consider a dedicated executor or structured concurrency design when work needs isolation; the convenient common pool may couple unrelated requests.

## TRY IT | Interview questions and model answers

### Why are streams lazy?

Laziness lets the terminal operation pull only needed elements, fuse stateless stages, and short-circuit. Stateful operations may still buffer substantial input.

### What is the difference between `map` and `flatMap`?

`map` produces one result value per input. `flatMap` maps each input to a stream and flattens all nested elements, supporting zero-to-many output.

### What makes a reduction safe in parallel?

Its operation is associative, its identity is neutral, accumulator and combiner are compatible, and behavioral functions do not share mutable state.

### Why should mutable reduction use `collect` rather than `reduce`?

Collector contracts explicitly provide separate accumulators and a combiner for mutable containers. Mutating a reduce identity can share state incorrectly and violates reduction assumptions.

### When is `parallelStream` appropriate?

For sufficiently large, splittable, CPU-bound data with independent operations and associative aggregation, after measurement under actual deployment load. It is not a general fix for blocking I/O.

### What does a spliterator contribute?

It traverses elements, can split remaining work into partitions, estimates size, and reports characteristics such as ordering, size, or distinctness that stream execution can exploit.

### Why prefer `orElseGet` sometimes?

`orElse` evaluates its argument eagerly even when the optional has a value. `orElseGet` evaluates its supplier only when empty.

## TRY IT | Exercises

- 1 Rewrite a parallel `forEach` that appends to an `ArrayList` as a safe collector. State order behavior.
- 2 Show that subtraction is not associative by partitioning four integers in two different ways.
- 3 Build a nested map of department to status to count using downstream collectors.
- 4 Implement a custom collector for count and `BigDecimal` total. Prove supplier, combiner, and finisher behavior.
- 5 Compare `findFirst` and `findAny` on ordered and unordered parallel sources.
- 6 Design a spliterator for an immutable array slice. State valid characteristics and a balanced `trySplit` rule.
- 7 Refactor code using `isPresent` plus `get` into `map`, `flatMap`, `or`, or `orElseThrow` where appropriate.
- 8 Analyze memory for `sorted().limit(10)` versus a size-10 heap on ten million records.

## RECAP | Chapter summary

Streams are lazy, single-use traversal pipelines rather than containers. Correct pipelines preserve non-interference, encounter-order requirements, and associative reduction laws. Collectors formalize mutable reduction and composition but require explicit duplicate, null, order, and mutability policies. `Optional` is a focused maybe-one return type, not a universal field or parameter wrapper. Spliterators expose traversal and partition properties; parallel speed depends on honest characteristics, balanced splitting, pure CPU work, and deployment context.

## TRY IT | Revision checklist

- ☐ I can classify stream operations as intermediate, terminal, stateful, or short-circuiting.
- ☐ I understand encounter order, `forEachOrdered`, `findFirst`, and `findAny`.
- ☐ I keep stream lambdas non-interfering and side-effect-free.
- ☐ I can state associativity and identity requirements for reduction.
- ☐ I can compose grouping and downstream collectors with explicit map policies.
- ☐ I know the mutability and null guarantees I may and may not infer from collection terminals.
- ☐ I use `Optional` lazily and at appropriate absence boundaries.
- ☐ I can explain spliterator splitting and characteristics.
- ☐ I evaluate parallel streams with correctness, source shape, workload, pool contention, and measurement.

## SDE-2 CORE CHAPTER

# Chapter 9 - Java I/O, NIO.2, Files, Buffers, and Serialization Boundaries

## Learning objectives

By the end of this chapter, you should be able to:

- choose byte, character, buffered, channel, and file APIs by data semantics;
- manage resources, partial operations, exceptions, charsets, and durability explicitly;
- use `Path` and `Files` without confusing lexical paths with filesystem identity;
- derive the `Buffer` position-limit-capacity state machine;
- explain blocking channels, selectors, direct buffers, and memory mapping at a contract boundary;
- perform safer temporary-file publication and directory traversal; and
- treat Java native serialization and all external formats as security boundaries.

## WHY IT WORKS | Why this matters at SDE-2

I/O connects Java's managed object world to files, sockets, pipes, and external data. Many production incidents originate at that boundary: leaked descriptors, default-charset corruption, partial writes, non-atomic publication, unbounded reads, path traversal, symlink races, or unsafe deserialization.

At SDE-2, you should reason beyond method names. What owns the resource? Can a read return fewer bytes? Is the output durable after `close`? Does moving a file replace atomically? Can input be trusted? Is text decoded incrementally? A robust design answers these questions and preserves them in tests and operational limits.

## WHY IT WORKS | First-principles model

I/O transfers finite sequences of bytes between a program and an external endpoint. Text is not bytes by itself; a charset maps characters to encoded bytes and back.

### TEXT

```
Java characters --CharsetEncoder--> bytes --external storage/network
Java characters <--CharsetDecoder-- bytes <--external storage/network
```

Classic I/O exposes sequential streams:

- `InputStream` and `OutputStream` transfer bytes.



- **Reader** and **Writer** transfer characters.
- decorators add buffering, data conversion, compression, or other behavior.

NIO exposes buffers and channels:

TEXT

```
channel <--> ByteBuffer <--> application
```

A buffer has three core indexes satisfying:

TEXT

```
0 <= position <= limit <= capacity
```

Write into a fresh buffer from **position** toward **limit**. After filling, **flip()** sets **limit** to the amount written and **position** to zero, preparing for reads. After consuming, **clear()** prepares the entire storage for overwrite; **compact()** preserves unread bytes and prepares the remaining suffix for more input.

**Specification boundary:** Java APIs define resource, buffer, path, channel, and serialization behavior. Operating-system caches, filesystem atomicity support, descriptor limits, direct-buffer allocation, selector implementation, and durability details vary by platform and JDK.

## Core terminology

- **Byte stream:** Sequence of raw 8-bit units.
- **Character stream:** Sequence decoded or encoded using a charset.
- **Decorator:** Wrapper that adds behavior while delegating to another stream.
- **Buffering:** Accumulating data to reduce calls and improve transfer efficiency.
- **Channel:** I/O connection supporting buffer-based operations.
- **Path:** Filesystem path representation; it may identify a nonexistent entry.
- **Partial read/write:** Successful operation transferring fewer units than requested.
- **Atomic move:** Move observed as one indivisible namespace update when supported.
- **Durability:** Extent to which data survives process or machine failure.
- **Direct buffer:** Buffer whose storage is intended to support efficient native I/O outside the ordinary Java heap array model.
- **Memory mapping:** Mapping a file region into virtual memory through a buffer-like view.
- **Serialization:** Encoding an object or data model into a transportable representation.

## Detailed mechanics

### Bytes, characters, and charsets

Use byte APIs for images, compressed data, protocols, hashes, and already encoded content. Use readers and writers for text. Always specify the charset at persistent or network boundaries, commonly

`StandardCharsets.UTF_8`. The platform default can differ across environments or be configured, and relying on it makes formats ambiguous.

Unicode characters may require multiple bytes, and a buffer boundary can split one encoded character.

`InputStreamReader` and `OutputStreamWriter` maintain encoding state correctly. Converting arbitrary chunks independently with `new String(chunk, UTF_8)` can corrupt a multibyte character split across chunks.

Malformed or unmappable input has a policy. Convenience decoding may replace invalid sequences. When strict validation matters, configure a `CharsetDecoder` with `CodingErrorAction.REPORT`.

### Stream layering and buffering

Streams compose:

JAVA

```
try (var in = new java.io.BufferedInputStream(
    new java.util.zip.GZIPInputStream(
        java.nio.file.Files.newInputStream(path))) {
    // consume decompressed bytes
}
```

The outermost close normally closes delegated resources, but check third-party wrappers. Buffering reduces calls across Java/native or network boundaries. It does not change asymptotic byte count, and wrapping an already buffered API repeatedly can waste memory.

`read()` returns an unsigned byte value `0..255` or `-1` for end-of-stream. Bulk `read(byte[])` can return any positive count up to array length, `-1` at end, and in some APIs zero under specified circumstances. Never assume one read fills a requested buffer. Similarly, a channel write can consume fewer remaining bytes, so loop until completion or until a nonblocking policy says to wait.

### Resource ownership and try-with-resources

Open streams, channels, directory streams, and file walks must be closed. Try-with-resources closes in reverse declaration order even when work throws. If both work and close throw, the work exception is primary and close exceptions are suppressed:

JAVA

```
catch (IOException ex) {
    for (Throwable suppressed : ex.getSuppressed()) {
        log(suppressed);
    }
}
```

Do not close a resource you merely borrow unless the API transfers ownership. A method accepting an `OutputStream` often should not close it; document whether it flushes. Conversely, a method that opens a path internally should normally own and close the opened resource.

`flush` asks buffered layers to push data downstream. It does not necessarily make bytes durable on physical storage. File-channel `force` requests stronger synchronization, but filesystem, device, and platform semantics still matter. Durable transactional publication may require syncing both file data and directory metadata according to platform-specific design.

## Path and Files

`Path` is a value-like representation of path components. It can be relative or absolute and does not imply existence. Useful distinctions:

- `normalize()` removes redundant lexical `.` and resolvable `..` segments without accessing the filesystem.
- `toAbsolutePath()` anchors a relative path but does not resolve symbolic links.
- `toRealPath()` accesses the filesystem, resolves links by default, and requires existence.
- `resolve` joins paths; if the argument is absolute, it may replace the base.
- `relativize` computes a lexical relative path between compatible paths.

Files methods offer creation, copying, moving, metadata, directory streams, buffered readers, and attributes. Check options precisely: for example, `CREATE_NEW` fails if the target already exists, while `TRUNCATE_EXISTING` destroys previous content once opening succeeds.

`Files.move` with `ATOMIC_MOVE` requests atomicity; providers may throw `AtomicMoveNotSupportedException`. Cross-file system moves generally cannot be one atomic rename. `REPLACE_EXISTING` controls collision behavior, but replacement details can vary. Design and test a fallback explicitly rather than silently weakening guarantees.

## Safe path containment and symbolic links

This check is only lexical:

JAVA

```
Path candidate = root.resolve(userInput).normalize();
if (!candidate.startsWith(root.normalize())) reject();
```

It blocks simple `..` traversal but does not defeat symlinks or validation-use races. Security-sensitive access needs a trusted root, link-aware operations, and a platform-specific threat model.

Directory traversal methods such as `Files.list`, `Files.walk`, and `Files.find` return streams backed by open resources and must be closed. Avoid following symbolic links unless cycle and trust behavior is deliberate. Never read an untrusted entire file with `readAllBytes` without a size limit; file size can change after checking.

## Buffer state transitions

For `ByteBuffer.allocate(8)`:

### TEXT

```
initial: position=0, limit=8, capacity=8
put 3 bytes: position=3, limit=8
flip: position=0, limit=3
get 2 bytes: position=2, limit=3, remaining=1
compact: unread byte moves to index 0; position=1, limit=8
```

`clear()` does not erase bytes; it resets indexes so storage can be overwritten. `rewind()` sets position to zero without changing limit, allowing rereading current content. `mark` records a position and `reset` returns to it when the mark remains valid. Relative gets advance position; absolute indexed gets do not.

Buffer overflow and underflow indicate state-machine errors: writing beyond remaining space or reading beyond the limit. `slice` and `duplicate` create buffers with independent position/limit state but shared underlying content. Mutation is visible through aliases. Read-only buffers reject writes but may still reflect shared storage changes.

Byte order matters for multi-byte numeric values. Network protocols often specify big-endian order. Set and document `ByteOrder` rather than relying on defaults or native order.

## Channels, nonblocking I/O, and selectors

File channels support positioned reads/writes, transfers, locking, mapping, and forcing. Socket channels can be blocking or nonblocking. In nonblocking mode, zero-byte progress can be a normal result, so a readiness loop registers channels with a `Selector` and reacts to acceptable, connectable, readable, or writable keys.

Readiness means an operation is likely to make progress, not that an entire message is available. Protocol framing must accumulate partial bytes across events. A selected key can become stale; handle cancellation, closure, and exceptions without spinning. Interest in write readiness should generally be enabled only while pending output exists, or a loop may wake continuously.

## Heap, direct, and mapped buffers

Heap buffers use managed memory and may expose an array. Direct buffers can reduce copying for native transfers, but allocation and reclamation are costlier and memory sits outside normal heap sizing. Mapped buffers expose file regions through virtual memory; page faults, truncation, flush, and unmapping are platform-sensitive. Neither `force` nor mapping creates an application transaction.

**HotSpot note:** Direct-buffer cleanup, native memory accounting, zero-copy transfer paths, selector providers, and mapped-buffer unmapping are implementation and OS sensitive. Do not depend on internal cleaner APIs.

## Serialization boundaries

Java native serialization reconstructs object graphs for types implementing `Serializable`. It captures private implementation shape, uses `serialVersionUID` for compatibility checks, supports custom hooks, and can invoke code during reconstruction. This makes it brittle for long-lived schemas and dangerous for untrusted data.

Never deserialize an untrusted native object stream merely because classes appear familiar. Apply process and protocol boundaries, strict allow-list filters where legacy serialization is unavoidable, size/depth/reference limits, and current security guidance. Prefer explicit schemas such as validated JSON, protocol buffers, or another designed wire format for service boundaries. Those formats also require input limits and safe parser configuration.

`transient` excludes a field from default serialization; it is not encryption. Constructors of ordinary serializable classes are not invoked in the usual way during reconstruction, so validate invariants in controlled reconstruction paths. A stable `serialVersionUID` prevents only certain version mismatch failures; it does not make schema evolution automatically correct or secure.

**Specification boundary:** Native serialization defines a protocol and hooks, but class evolution has limited compatibility rules. It is not a general-purpose safe RPC format and should be treated as a legacy trust boundary.

## TRACE IT | Worked Java example

This method writes UTF-8 text to a sibling temporary file, forces file content, and attempts atomic publication:

JAVA (continued 1/2)

```
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.charset.StandardCharsets;
import java.nio.file.AtomicMoveNotSupportedException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.StandardOpenOption;

final class AtomicTextFile {
    static void replace(Path target, Iterable<String> lines) throws IOException {
        Path absolute = target.toAbsolutePath();
        Path parent = absolute.getParent();
        if (parent == null) throw new IOException("target has no parent");
        Files.createDirectories(parent);

        Path temp = Files.createTempFile(parent, ".pending-", ".tmp");
        boolean published = false;
        try {
            try (FileChannel channel = FileChannel.open(
                temp, StandardOpenOption.WRITE,
                StandardOpenOption.TRUNCATE_EXISTING)) {
                for (String line : lines) {
                    if (line.indexOf('\n') >= 0 || line.indexOf('\r') >= 0) {
                        throw new IllegalArgumentException("line contains a newline");
                    }
                }
            }
            published = true;
            Files.move(temp, absolute, StandardCopyOption.REPLACE_EXISTING);
        } catch (IOException e) {
            if (published) {
                Files.delete(temp);
            }
            throw e;
        }
    }
}
```

## JAVA (continued 2/2)

```

        }
        byte[] encoded = (line + "\n").getBytes(StandardCharsets.UTF_8);
        ByteBuffer buffer = ByteBuffer.wrap(encoded);
        while (buffer.hasRemaining()) {
            channel.write(buffer);
        }
    }
    channel.force(true);
}

try {
    Files.move(temp, absolute,
        StandardCopyOption.ATOMIC_MOVE,
        StandardCopyOption.REPLACE_EXISTING);
} catch (AtomicMoveNotSupportedException unsupported) {
    // This fallback preserves complete-file publication but not atomic replacement.
    Files.move(temp, absolute, StandardCopyOption.REPLACE_EXISTING);
}
published = true;
} finally {
    if (!published) {
        Files.deleteIfExists(temp);
    }
}
}
}
}

```

Creating the temporary file in the target directory improves the chance that source and target share a filesystem, which is usually necessary for atomic rename. The fallback is an explicit weakening; a system that requires atomicity should propagate failure instead. The example does not claim full crash durability of directory metadata on every platform.

## TRACE IT | Execution or memory walkthrough

For lines **alpha** and **beta**:

- 1 A unique temp file is created in the target directory.
- 2 **alpha**\n encodes to six UTF-8 bytes and **ByteBuffer.wrap** starts with **position=0**, **limit=6**, **capacity=6**.
- 3 Each channel write advances position by the actual count. The loop handles partial writes until **position == limit**.
- 4 The same process writes **beta**\n. Each encoded byte array becomes unreachable after its iteration.
- 5 **force(true)** requests synchronization of content and metadata for the file channel.
- 6 Closing the channel releases its descriptor.
- 7 Atomic move is attempted. Before the namespace update, readers see the old target; after it, they see the complete new target under supporting provider semantics.
- 8 If writing or moving fails, the **finally** block attempts to remove the temporary file. The original target is not truncated by the write phase.

This design uses memory proportional to the largest encoded line, not the whole file. It still permits an arbitrarily large single line and an infinite iterable; callers may need explicit byte and line limits.

## Complexity and performance

Writing `b` encoded bytes is  $O(b)$  time and uses  $O(\text{maxLineBytes})$  temporary memory in the example. Filesystem and device latency dominate CPU complexity. Buffering changes call counts and constants, not the amount of data.

| Task                            | Typical time                      | Important caveat                         |
|---------------------------------|-----------------------------------|------------------------------------------|
| sequential read/write           | $O(\text{bytes})$                 | partial operations and device latency    |
| path <code>normalize</code>     | $O(\text{components})$            | lexical only                             |
| directory traversal             | $O(\text{entries})$               | metadata calls, symlinks, open resources |
| copy or move across filesystems | $O(\text{bytes})$                 | cannot usually be atomic                 |
| same-filesystem rename          | often near $O(1)$ namespace work  | provider-dependent atomicity/durability  |
| buffer flip/clear               | $O(1)$                            | clear does not erase content             |
| buffer compact                  | $O(\text{unread bytes})$          | moves retained content                   |
| memory map access               | $O(\text{bytes touched})$ logical | page faults and OS caching dominate      |

Many tiny reads and writes pay syscall, locking, TLS, or device overhead repeatedly. Choose buffer sizes empirically. Huge buffers increase latency to first processing, direct-memory use, and concurrent footprint. Zero-copy channel transfers can help large file movement on supported systems but can fall back internally.

## WATCH OUT | Edge cases and common mistakes

- Relying on the platform default charset for stored or transmitted text.
- Assuming one read fills an array or one write drains a buffer.
- Forgetting that `read` uses `-1` for end-of-stream.
- Leaking streams returned by `Files.list` or `Files.walk`.
- Calling `readAllBytes` on unbounded input.
- Confusing `normalize`, absolute path, and real path.
- Treating a lexical `startsWith` containment check as symlink-safe authorization.
- Assuming `ATOMIC_MOVE` is supported or silently weakening a required guarantee.
- Believing `flush` or `close` alone guarantees crash durability.
- Calling `clear()` and assuming sensitive bytes were erased.
- Forgetting `flip()` before reading data just written to a buffer.
- Losing partial protocol frames between nonblocking reads.
- Enabling selector write interest continuously and causing a busy loop.
- Allocating unbounded direct buffers and watching only Java heap metrics.
- Mutating shared storage through a sliced or duplicated buffer alias.
- Deserializing native Java objects from untrusted input.
- Treating `transient` as confidentiality or `serialVersionUID` as safe evolution.

## Production engineering notes

Every I/O entry point needs limits: maximum bytes, lines, nesting, files, traversal depth, open handles, concurrency, and time. Use streaming parsing rather than whole-file loading for large content. Reject overlong records before allocating proportional memory.

Make ownership explicit. The layer that opens a resource should normally close it. When APIs accept caller-owned streams, state close and flush behavior. Propagate interruption and cancellation through blocking workflows; configure socket connect/read deadlines and handle stalled peers.

Use temp-write plus rename for complete-file publication, with a documented policy when atomic move is unavailable. Separate atomic visibility from crash durability; use a storage engine when durable transactions are required.

Protect endpoints with least-privilege permissions, trusted roots, generated names, and symlink-aware operations. Validate archive paths and decompression ratios.

Measure bytes, latency, open handles, failures, direct memory, and rejected input. Avoid logging payloads. Isolate legacy native serialization with narrow filters, graph limits, an allow-list, and a migration plan.



## TRY IT | Interview questions and model answers

### When do you use a Reader instead of an InputStream?

Use a reader when the data is text and must be decoded with a known charset. Use an input stream for raw bytes. An `InputStreamReader` bridges them while handling multibyte boundaries.

### Why must channel writes be in a loop?

A successful write may consume fewer than all remaining bytes, especially for sockets or nonblocking channels. Advance is reflected in buffer position, so loop while `hasRemaining`, subject to readiness and timeout policy.

### Explain flip, clear, and compact.

`flip` changes from filling to draining by setting limit to current position and position to zero. `clear` prepares all storage for overwrite without erasing it. `compact` preserves unread bytes, moves them to the front, and prepares the remaining area for more input.

### Is `Path.normalize()` sufficient to prevent path traversal?

No. It is a lexical operation. It can reject simple `..` escapes but does not resolve symlinks or remove validation-use races. Security-sensitive access needs a trusted-root and filesystem-aware strategy.

### Does try-with-resources lose close exceptions?

If work throws, close exceptions are attached as suppressed exceptions to the primary failure. Resources close in reverse declaration order.

### Why is Java native deserialization dangerous?

It reconstructs graphs and can invoke class-specific code from attacker-controlled stream content. Crafted graphs can abuse gadget paths or exhaust resources. Do not use it for untrusted data; tightly filter and bound unavoidable legacy uses.

### Do virtual threads replace NIO?

They make blocking-style concurrency practical for many workloads, but do not remove protocol framing, timeouts, capacity, partial I/O, or the value of event loops in specialized systems.

## TRY IT | Exercises

- 1 Write a byte-copy loop that handles partial reads and writes with a reusable `ByteBuffer`. Dry-run its indexes for a split read.
- 2 Decode UTF-8 using a strict `CharsetDecoder` and reject malformed input.
- 3 Compare `normalize`, `toAbsolutePath`, and `toRealPath` for a path containing `..` and a symbolic link.
- 4 Implement a bounded line reader that rejects more than one megabyte or ten thousand lines.
- 5 Modify the worked example so atomic move support is mandatory. Explain the operational consequence.
- 6 Design a nonblocking length-prefixed message decoder that preserves incomplete header and body bytes across reads.
- 7 Threat-model archive extraction, including traversal, symlinks, entry count, and decompression bombs.
- 8 Propose a migration plan from a native serialized cache to a versioned explicit schema.

## RECAP | Chapter summary

Java I/O begins with byte semantics, explicit character encoding, partial transfer, and resource ownership. Classic streams compose decorators; NIO channels move data through buffers governed by position, limit, and capacity. `Path` is a representation, while filesystem identity, symlinks, atomic moves, and durability are provider concerns. Direct and mapped buffers trade heap copying for native lifecycle complexity. Every external format is a trust boundary, and native Java deserialization is especially risky. Production I/O must be bounded, observable, charset-explicit, and honest about atomicity and durability.

## TRY IT | Revision checklist

- ☐ I choose byte versus character APIs and always define external charsets.
- ☐ I handle partial reads/writes, EOF, buffering, and close ownership.
- ☐ I understand suppressed exceptions in try-with-resources.
- ☐ I distinguish lexical, absolute, and real paths.
- ☐ I can explain atomic visibility versus crash durability.
- ☐ I can dry-run `position`, `limit`, and `capacity` through flip, clear, rewind, and compact.
- ☐ I understand shared content in buffer slices and duplicates.
- ☐ I can explain selector readiness and partial protocol framing.
- ☐ I monitor direct/native memory as well as heap.
- ☐ I reject unbounded input and unsafe native deserialization.

## SDE-2 CORE CHAPTER

# Chapter 10 - Collection Complexity and Selection Matrix

Complexities below describe common OpenJDK implementations for ordinary workloads, not interface-level guarantees unless stated. Hash-table operations are expected-time claims under an adequate hash distribution. Adversarial keys, resizing, comparator cost, allocation, cache locality, and concurrency can dominate a real service.

## List implementations

| Operation                                   | <code>ArrayList</code>        | <code>LinkedList</code>         | Engineering note                                     |
|---------------------------------------------|-------------------------------|---------------------------------|------------------------------------------------------|
| indexed <code>get</code> / <code>set</code> | $O(1)$                        | $O(n)$                          | Array locality usually favors <code>ArrayList</code> |
| append                                      | amortized $O(1)$              | $O(1)$                          | Array resize occasionally copies elements            |
| prepend                                     | $O(n)$                        | $O(1)$                          | <code>ArrayDeque</code> is usually the better queue  |
| insert/remove by index                      | $O(n)$ shift                  | $O(n)$ traversal, $O(1)$ relink | Traversal often dominates linked-list edits          |
| search by value                             | $O(n)$                        | $O(n)$                          | Equality cost is part of the constant                |
| iteration                                   | $O(n)$                        | $O(n)$                          | Pointer chasing can hurt locality                    |
| memory per element                          | reference plus spare capacity | node object plus links          | Measure retained size for large collections          |

Default choice: `ArrayList`. Choose `LinkedList` only when its list-plus-deque contract and a measured mutation pattern justify node overhead. Even frequent insertion is not sufficient if finding the insertion point is linear.

`CopyOnWriteArrayList` makes traversal stable and unsynchronized for readers by copying the backing array on every structural mutation. It fits small, read-mostly listener/configuration sets; it is disastrous for frequent writes or large lists.

## Set and map implementations

| Type                                                       | Lookup / update                           | Ordering                                 | Null policy                                     | Representative use                                 |
|------------------------------------------------------------|-------------------------------------------|------------------------------------------|-------------------------------------------------|----------------------------------------------------|
| <code>HashMap</code>                                       | expected $O(1)$                           | unspecified                              | one null key, null values                       | general mutable map                                |
| <code>LinkedHashMap</code>                                 | expected $O(1)$                           | insertion or access order                | like <code>HashMap</code>                       | predictable traversal, bounded LRU building block  |
| <code>TreeMap</code>                                       | $O(\log n)$                               | comparator/natural sorted                | null key generally rejected by natural ordering | range queries, nearest-key navigation              |
| <code>EnumMap</code>                                       | $O(1)$ array-like                         | enum declaration order                   | null key rejected                               | dense enum-keyed state                             |
| <code>IdentityHashMap</code>                               | expected $O(1)$                           | unspecified                              | permits null                                    | graph/topology algorithms using identity semantics |
| <code>ConcurrentHashMap</code>                             | expected $O(1)$                           | unspecified, weakly consistent traversal | null keys/values rejected                       | shared concurrent lookup/update                    |
| <code>immutableMap.of()</code> / <code>Map.copyOf()</code> | implementation-dependent efficient lookup | unspecified                              | null rejected                                   | fixed snapshots and safe sharing                   |

Corresponding sets are commonly backed by maps or tree structures:

| Type                                       | Membership                          | Ordering                      | Notes                                   |
|--------------------------------------------|-------------------------------------|-------------------------------|-----------------------------------------|
| <code>HashSet</code>                       | expected $O(1)$                     | unspecified                   | mutable elements can become unfindable  |
| <code>LinkedHashSet</code>                 | expected $O(1)$                     | insertion order               | extra link metadata                     |
| <code>TreeSet</code>                       | $O(\log n)$                         | sorted                        | comparator equality controls uniqueness |
| <code>EnumSet</code>                       | $O(1)$ bit-vector-like              | enum order                    | exceptionally compact for enum domains  |
| <code>CopyOnWriteArraySet</code>           | $O(n)$ lookup, $O(n)$ copy on write | insertion-like snapshot order | very small read-mostly sets             |
| <code>ConcurrentHashMap.newKeySet()</code> | expected $O(1)$                     | weakly consistent             | scalable concurrent membership set      |

Important `HashMap` facts:

- Capacity and load factor affect resizing, memory, and traversal cost.

- Current OpenJDK **HashMap** implementations may transform a collision-heavy bin into a tree when thresholds and table capacity permit. That is an implementation strategy, not permission to use poor keys.
- **computeIfAbsent** is not a general exactly-once side-effect facility. Read each map's contract, keep mapping functions short, and avoid recursive structural updates.
- Mutating an equality/hash field after insertion violates the lookup assumption even though the entry still occupies a bucket.
- Iteration order can look stable in a small experiment and still have no contract.

## Queues, dequeues, and heaps

| Type                         | Add/remove               | Head access                                   | Capacity / blocking          | Use                                                     |
|------------------------------|--------------------------|-----------------------------------------------|------------------------------|---------------------------------------------------------|
| <b>ArrayDeque</b>            | amortized $O(1)$ at ends | $O(1)$                                        | unbounded, nonblocking       | default stack, queue, deque                             |
| <b>PriorityQueue</b>         | $O(\log n)$ add/remove   | $O(1)$ peek                                   | unbounded, nonblocking       | next minimum/maximum by comparator                      |
| <b>ConcurrentLinkedQueue</b> | expected $O(1)$          | typically $O(1)$ ; may traverse removed nodes | unbounded, nonblocking       | scalable multi-producer/consumer queue                  |
| <b>ArrayBlockingQueue</b>    | $O(1)$                   | $O(1)$                                        | bounded, blocking            | fixed-capacity back pressure                            |
| <b>LinkedBlockingQueue</b>   | $O(1)$                   | $O(1)$                                        | optionally bounded, blocking | producer/consumer, usually explicit bound               |
| <b>PriorityBlockingQueue</b> | $O(\log n)$              | $O(1)$                                        | unbounded, blocking take     | concurrent priority scheduling without capacity control |
| <b>SynchronousQueue</b>      | handoff                  | none stored                                   | zero capacity, blocking      | direct producer-consumer rendezvous                     |
| <b>DelayQueue</b>            | $O(\log n)$              | eligible head                                 | unbounded                    | delayed/expiry work                                     |

Use **offer**, **poll**, and **peek** when absence/capacity is expected and should be represented by a result. Use **add**, **remove**, and **element** when failure should be exceptional. Blocking queues add **put** and **take**; timed variants let cancellation and service-level policy participate.

**PriorityQueue** guarantees that **peek/remove** exposes a least element according to its ordering. Its iterator is not sorted. To emit sorted order, repeatedly remove elements from a copy, or sort a collection explicitly.

## Sorted and navigable operations

`NavigableMap` and `NavigableSet` support relative queries in  $O(\log n)$  for tree implementations:

- `lower`: greatest element strictly less than the key.
- `floor`: greatest element less than or equal to the key.
- `ceiling`: least element greater than or equal to the key.
- `higher`: least element strictly greater than the key.
- `subMap`, `headMap`, and `tailMap`: usually backed range views.

Backed views enforce their range. An insertion outside the view range fails. Changes through either the view or the source are visible through the other, subject to the collection's synchronization and iterator contracts.

## Views, wrappers, and copies

These APIs have deliberately different ownership semantics:

| API                                             | Structural mutability                | Backing relationship             | Null handling                                |
|-------------------------------------------------|--------------------------------------|----------------------------------|----------------------------------------------|
| <code>Arrays.asList(array)</code>               | fixed size; <code>set</code> allowed | backed by array                  | permits null                                 |
| <code>list.subList(a, b)</code>                 | generally mutable within range       | backed by list                   | source policy                                |
| <code>Collections.unmodifiableList(list)</code> | wrapper rejects mutation             | reflects backing-list changes    | source policy                                |
| <code>List.copyOf(source)</code>                | unmodifiable                         | snapshot-like shallow copy/reuse | rejects null                                 |
| <code>List.of(values...)</code>                 | unmodifiable                         | independent fixed value          | rejects null                                 |
| <code>stream.toList()</code>                    | unmodifiable                         | result collection                | may contain null elements produced by stream |
| <code>Collectors.toList()</code>                | no mutability/type guarantee         | new result                       | collector behavior                           |

Unmodifiable is not deeply immutable. If elements are mutable, their state can change. A defensive copy protects collection structure only; copy elements or use immutable value types when deeper isolation is required.

## Iterator consistency

Iterator labels describe detection and visibility, not thread safety of arbitrary compound actions:

- Fail-fast iterators on many ordinary collections may throw `ConcurrentModificationException` after an uncoordinated structural change. Detection is best-effort, not a synchronization mechanism.
- Snapshot iterators, such as those of copy-on-write collections, traverse a stable historical array and do not see later writes.
- Weakly consistent iterators on concurrent collections tolerate concurrent updates and may reflect some updates without throwing. They do not freeze a transactionally consistent snapshot.

Use external locking when multiple operations must preserve one invariant and the collection does not supply an atomic compound method. `if (!map.containsKey(k)) map.put(k, v)` is a race on a shared map; use `putIfAbsent`, `compute`, or explicit coordination as the required semantics dictate.

## Stream and collector cost reminders

A stream pipeline is not a collection. It describes one traversal and is normally single-use. Complexity is the sum of traversal and operations:

| Operation                              | Typical cost                  | Important qualification                                                                                              |
|----------------------------------------|-------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <code>filter</code> , <code>map</code> | $O(n)$                        | function cost and boxing may dominate                                                                                |
| <code>distinct</code>                  | expected $O(n)$               | retains seen values; ordered mode can cost more                                                                      |
| <code>sorted</code>                    | $O(n \log n)$                 | buffers elements; comparator cost matters                                                                            |
| <code>limit</code>                     | passes at most $k$ downstream | upstream filters or stateful stages can still perform $O(n)$ work; ordered parallel pipelines may coordinate heavily |
| <code>findFirst</code>                 | short-circuiting              | encounter order constrains parallel execution                                                                        |
| grouping                               | expected $O(n)$               | retains keys and downstream accumulation state                                                                       |
| <code>toMap</code>                     | expected $O(n)$               | merge policy required when duplicate output keys are possible                                                        |

Collector characteristics such as `CONCURRENT`, `UNORDERED`, and `IDENTITY_FINISH` are promises to the reduction framework. A mutable result container alone does not make a collector concurrent.

## Selection questions

Choose a collection by answering in order:

- 1 What semantics are required: sequence, uniqueness, association, priority, range navigation, or handoff?
- 2 Is order absent, insertion-based, access-based, sorted, or merely head-priority?
- 3 Which operations dominate, and what are their input sizes?
- 4 Is mutation local, externally synchronized, or concurrent?
- 5 Must a compound invariant span several operations?
- 6 Are nulls valid domain values, and can the chosen type represent them?
- 7 Who owns the collection, and should the API expose a view, wrapper, or copy?
- 8 What bounds memory: maximum size, admission control, expiry, or back pressure?
- 9 Does iteration need a snapshot, weak consistency, or strict external coordination?
- 10 Have representative allocation, locality, contention, and tail latency been measured?



## Rapid decision matrix

The strongest interview answer states the contract first, compares two plausible choices, names the dominant workload operation, and closes with ownership, concurrency, and measurement implications.

| Requirement                      | Start with                                         | Reconsider when                                                              |
|----------------------------------|----------------------------------------------------|------------------------------------------------------------------------------|
| general indexed sequence         | <code>ArrayList</code>                             | frequent measured front edits or immutable persistent structure needed       |
| stack / FIFO / double-ended work | <code>ArrayDeque</code>                            | cross-thread blocking or bounded capacity required                           |
| general key lookup               | <code>HashMap</code>                               | sorted/range operations, stable order, enum keys, or concurrency required    |
| predictable insertion traversal  | <code>LinkedHashMap</code>                         | sorted key semantics required                                                |
| sorted map and nearest-key query | <code>TreeMap</code>                               | hash lookup is sufficient and ordering cost is wasted                        |
| enum membership                  | <code>EnumSet</code>                               | domain is not an enum                                                        |
| minimum/maximum scheduling       | <code>PriorityQueue</code>                         | full sorted iteration or bounded blocking is required                        |
| shared concurrent map            | <code>ConcurrentHashMap</code>                     | one invariant spans external state or several keys                           |
| bounded producer/consumer        | <code>ArrayBlockingQueue</code>                    | transfer semantics, priority, or different throughput characteristics matter |
| fixed public result              | <code>List.copyOf</code> / <code>Map.copyOf</code> | caller intentionally needs a live view                                       |
| small read-mostly listener set   | copy-on-write collection                           | writes or collection size grow                                               |

**Part II - Practice and Handoff**

# Practice Ladder and Next Steps

**TRY IT | Practice ladder**

- Foundation: collection and file API recall
- Interview Core: choose representations and pipelines
- SDE-2 Follow-up: aliasing, parallelism, and resource ownership
- Challenge: design a bounded high-volume data path

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

## Navigation

[Previous volume: 18B - Advanced Java B: Language, OOP, and Modern Java](#)

[Series index](#)

[Next volume: 18D - Advanced Java D: Concurrency and the Memory Model](#)

**TRY IT | Completion check**

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

# Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

| STEP | FOCUSED LEARNING PATH                                         |
|------|---------------------------------------------------------------|
| 1    | Java Foundations for Problem Solving                          |
| 2    | Time and Space Complexity                                     |
| 3    | Number Systems and Math Foundations for DSA Interviews        |
| 4    | Bit Manipulation in Java                                      |
| 5    | Loop Mastery, Patterns, and Index Calculations                |
| 6    | Arrays and Array Problem-Solving Patterns                     |
| 7    | Strings and String Problem-Solving Patterns                   |
| 8    | Hashing: Maps, Sets, Frequency, and Prefix State              |
| 9    | Recursion and Backtracking in Java                            |
| 10   | Linked Lists: Pointer Reasoning and Mutation                  |
| 11   | Stacks, Queues, Deques, and Monotonic Patterns                |
| 12   | Binary Search: Bounds, Answers, and Invariants                |
| 13   | Trees, Binary Search Trees, and Tries                         |
| 14   | Heaps, Priority Queues, Selection, and Top-K                  |
| 15   | Graphs: Traversal, Ordering, Paths, and Union-Find            |
| 16   | Greedy Algorithms: Recognition, Proofs, and Scheduling        |
| 17   | Dynamic Programming: State, Transitions, and Optimization     |
| 18   | Advanced Java Engineering and the SDE-2 Interview (Parts A-J) |

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

# About the Author

## Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

## Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

## Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

## Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

**Connect:** [LinkedIn](#) | [GitHub](#)

# Copyright and Publishing Notes

---

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 18 of 18 - Part C of J. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.